Project Report On

CORE-ALIGN : Real Time Video Based Posture Evaluation System

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science Engineering

By

Aaron Joseph Manikutty (U2203004)
Abin J Arackal (U2203013)
Angelo Varghese (U2203043)
Bijay Krishna D (U2203068)

Under the guidance of

Ms. Dincy Paul

Computer Science Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039

March 2026

CERTIFICATE

This is to certify that the project report entitled ***"CORE-ALIGN : Real Time Video Based Posture Evaluation System"*** *is a bonafide record of the work done by* ***Aaron Joseph Manikutty (U2203004)****,* ***Abin J Arackal (U2203013)****,* ***Angelo Varghese (U2203043)****,* ***Bijay Krishna D (U2203068)*** *submitted to the Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in "Computer Science Engineering" during the academic year 2025-2026.*

Project Guide:
Ms. Dincy Paul
Assistant Professor
Dept. of CSE
RSET

Project Co-ordinator:
Dr. Jisha G.
Associate Professor
Dept. of CSE
RSET

Head of the Department:
Dr. Preetha K.G.
Professor
Dept. of CSE
RSET

ACKNOWLEDGMENT

We wish to express our sincere gratitude towards **Fr. Dr. Jaison Paul Mulerikkal**, Principal of RSET, and **Dr. Preetha K.G.**, Head of the Department of Computer Science and Engineering for providing us with the opportunity to undertake our project, "CORE-ALIGN : Real Time Video Based Posture Evaluation System".

We are highly indebted to our project coordinators, **Dr. Jisha G.**, Associate Professor, Department of Computer Science and Engineering and **Dr. Preetha K.G.**, Head Of Department, Department of Computer Science and Engineering, for their valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Ms. Dincy Paul** for her patience and all the priceless advice and wisdom she has shared with us.

Last but not the least, We would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Aaron Joseph Manikutty

Abin J Arackal

Angelo Varghese

Bijay Krishna D

Abstract

This project presents CORE-ALIGN, a real time video based posture evaluation system designed to analyze exercise movements and provide corrective feedback during workout sessions. The system uses computer vision and machine learning techniques to extract human skeletal motion from video input and convert it into joint angle representations. A multi-stage motion analysis pipeline is employed, consisting of motion acquisition, signal conditioning, repetition segmentation, and feedback generation.

Pose estimation is performed using MediaPipe Pose to obtain body landmarks, which are transformed into angular motion signals for robust analysis. A temporal deep learning model is used to identify repetition boundaries and analyze movement patterns. The system evaluates user performance by comparing joint angle trajectories with a trainer reference and computes metrics such as posture accuracy, range of motion, and execution speed. Based on these metrics, corrective feedback is generated in real time.

The platform is implemented as a web application with gesture interaction for hands free control and uses cloud services for user management and data storage. Experimental results demonstrate the effectiveness of the system in providing accurate and real time posture evaluation. The proposed approach offers a scalable and accessible solution to improve exercise quality in home fitness environments.

Contents

List of Abbreviations

Acronym	**Expansion**
HAR	Human Activity Recognition
IMU	Inertial Measurement Unit
ML	Machine Learning
MAE	Mean Absolute Error
RMSE	Root Mean Square Error
ROM	Range of Motion
BiLSTM	Bidirectional Long Short-Term Memory
RGB	Red Green Blue
NoSQL	Not Only Structured Query Language
GPU	Graphics Processing Unit
VRAM	Video Random Access Memory
GTX	Giga Texel Shader eXtreme

List of Figures

List of Tables

Chapter 1

Introduction

The recent growth of digital technologies and artificial intelligence have changed the way people approach health and fitness. Online workout programs along with home workout routines have become increasingly popular due to their ease of use and accessibility. However, one of the main challenges associated with unsupervised workouts is maintaining correct posture and exercise form. Without direct guidance from a professional trainer, users may unknowingly perform exercises incorrectly, which can reduce the effectiveness of the workout and increase the risk of bodily injury.

Recent advancements in computer vision and artificial intelligence have made it possible to analyze human body movements using cameras. Pose estimation technologies can extract key body landmarks from video streams and enable automated analysis of human posture and movement. This opens up new opportunities for developing intelligent fitness systems that can provide real time feedback and assist users during workouts.

This project proposes a web based posture analysis system that compares a user's live exercise posture with that of a trainer's video using computer vision techniques. The system will then utilize real time pose estimation to detect body landmarks and then evaluate posture accuracy. By providing immediate feedback and performance summaries after each session, the platform aims to improve exercise safety, enhance training effectiveness, and make guided fitness training more accessible. The following sections of this chapter discuss the background of the problem, the motivation behind the project, the objectives and challenges involved and the overall organization of the report.

1.1 Background

Poor posture during physical activities increases the risk of injury and reduces training effectiveness. To address this, a web based AI system compares a user's live posture with trainer videos, enabling accurate form assessment. The system provides real time alerts for posture corrections during the workout and delivers a detailed performance summary after each session to support continuous improvement. .

1.2 Problem Definition

Maintaining proper form during home workouts is difficult without immediate professional feedback, and this lack of guidance increases the risk of injury while reducing overall training effectiveness. Additionally, there is a clear technology gap: current virtual systems cannot perform real time posture comparisons between a user and a trainer using standard video alone, as they still rely on wearable sensors.

1.3 Scope and Motivation

The system performs real time pose estimation using a desktop webcam, allowing it to continuously monitor and analyze the user's body movements during a workout session. At the same time, it captures trainer videos directly from the screen to perform pose detection on both the user and the trainer simultaneously. This enables live pose comparison, where the system evaluates alignment and movement accuracy frame by frame. Whenever a deviation in posture is detected, feedback is generated instantly to help the user correct their form without interrupting the workout.

After each session, the system provides comprehensive post session feedback, including accuracy scores and visual representations of performance to help users clearly understand their strengths and areas for improvement. Additionally, users have access to a personal statistics dashboard that monitors performance trends over time. All of these features are integrated into a fully web based interface designed specifically for desktop browsers, making the platform easily accessible.

1.4 Objectives

- Develop a user friendly web application for user management, workout selection, and live training sessions.
- Create a computer vision engine to instantly compare the user's live posture against a

trainer's video.

- Deliver immediate feedback during workouts to correct the user's form as mistakes happen.
- Generate a detailed performance summary after each session, highlighting achievements and areas for improvement.
- Implement personalized dashboards for different users to track their long term progress and historical data.

1.5 Challenges

The development of the proposed posture evaluation system involved several challenges related to data, model training, and real-time system integration.

One of the primary challenges was the absence of a suitable publicly available dataset for exercise repetition analysis. As a result, a custom dataset had to be created manually using recorded videos and external sources, which required significant effort in data collection, preprocessing, and annotation.

Another major challenge was the long training time of the model. Due to the temporal nature of the data and the complexity of the CNN-BiLSTM architecture, training required substantial computational resources and time. Iterative experimentation with hyperparameters and model configurations further increased the development time.

System testing also posed significant challenges, particularly in real-time scenarios. Ensuring consistent performance across different lighting conditions, camera angles, and user variations required extensive testing. Additionally, debugging issues in a multi component system involving frontend, backend, and real time communication made the testing process more complex.

The system also faced challenges related to pose estimation reliability. Occlusions, multiple people in the frame, and sudden motion changes affected the accuracy of joint detection, which in turn impacted downstream analysis.

1.6 Assumptions

- MediaPipe provides sufficiently accurate and temporally stable 3D body keypoints for joint-angle computation.
- The trainer video represents the reference (ideal) execution for each exercise.
- The user performs the exercise under reasonably controlled and acceptable conditions, including adequate lighting, appropriate camera placement, and a stable posture setup.
- The full body of both the trainer and the user is visible with minimal self occlusion and no major external occlusions.

1.7 Societal / Industrial Relevance

The proposed system has significant societal relevance as it promotes safer and more effective physical excercise in an increasingly digital fitness environment. With the growing popularity of home workouts and online fitness programs, many individuals exercise without direct supervision from professional trainers. This will often lead to incorrect posture and movement, which can increase the risk of injuries and reduce the effectiveness of training. By providing real-time posture analysis and corrective feedback, the system helps users maintain proper form and perform exercises safely.

Additionally, the platform improves accessibility to quality fitness guidance by allowing users to receive posture feedback using only a standard webcam and a web browser, without requiring expensive wearable sensors or specialized equipment. This makes posture monitoring more widely available to a broader population. The system also encourages longnterm health and fitness by enabling users to track their progress through personalized dashboards and performance summaries. Overall, the solution supports healthier lifestyles, reduces injury risks during unsupervised workouts, and leverages technology to make guided fitness training more accessible and scalable for society.

1.8 Organization of the Report

This report is organized into five chapters that describe the development and evaluation of the proposed posture analysis system.

- **Chapter 1: Introduction**

  This chapter presents an overview of the project, including the background of the problem, motivation, objectives, and challenges involved in developing an AI-based posture analysis system. It also introduces the significance of using computer vision techniques for improving workout posture and safety.

- **Chapter 2: Literature Survey**

  This chapter reviews existing research and technologies related to posture detection, pose estimation, and computer vision based fitness monitoring systems. It analyzes previous approaches and highlights the need for an intelligent system capable of providing real time posture feedback during workouts.

- **Chapter 3: System Design**

  This chapter explains the overall architecture and design of the proposed system. It describes the modules involved in the posture analysis platform, the technologies used, and the workflow of the system including pose estimation, posture comparison, and feedback generation.

- **Chapter 4: Results and Discussion**

  This chapter presents the implementation results of the system and discusses the performance of the posture analysis model. It includes observations obtained during testing and explains how the system evaluates and provides feedback on user posture.

- **Chapter 5: Conclusion**

  This chapter summarizes the key outcomes of the project and highlights the effectiveness of the proposed posture analysis system. It also discusses possible improvements and future enhancements that can further improve the system's accuracy and usability.

1.9 Summary of Chapter

This chapter introduced the fundamental concepts and the motivations behind the proposed posture analysis system. It discussed the background of home based fitness training and highlighted the challenges associated with maintaining correct posture without professional supervision. The problem definition, scope, objectives, and key challenges of the project were presented to outline the goals of the proposed solution. In addition, the assumptions and societal relevance of the system were discussed to emphasize its potential impact in promoting safer and more effective workouts. Finally, the overall organization of the report was described to provide a clear structure for the chapters that follow.

Chapter 2

Literature Survey

2.1 Introduction

Human Activity Recognition and automated exercise monitoring have gained significant attention in recent years due to the increasing demand for intelligent fitness tracking and rehabilitation systems. With advancements in computer vision and machine learning, researchers have developed various approaches to detect human movements, recognize exercise types, and count repetitions automatically. These systems aim to assist users in maintaining proper workout routines, improving training efficiency, and reducing the need for manual supervision. Many recent studies have explored the use of deep learning models, skeleton based pose estimation techniques, and wearable sensor data to analyze human motion patterns. Such methods enable the detection of repetitive movements and facilitate the automatic counting of exercise repetitions in real time.

Several works have proposed different techniques for exercise recognition and repetition counting. Vision based approaches commonly utilize pose estimation algorithms to extract skeletal joint coordinates from video data, allowing systems to analyze motion patterns and detect repetitive movements. Deep learning models are often applied to capture temporal dependencies in human motion, improving the accuracy of exercise classification and repetition detection. In addition, sensor based methods use inertial measurement units attached to the body to collect motion signals and provide highly accurate activity recognition. While these methods have shown promising results, each approach presents its own advantages and limitations in terms of accuracy, computational requirements, and usability.

2.1.1 Recognition and Repetition Counting for Complex Physical Exercises Using Deep Learning

This paper [1] proposes a deep learning based framework for recognizing physical exercises and automatically counting repetitions. The system analyzes temporal patterns in human motion captured from video data to identify cyclic movements such as push ups and squats. A neural network model is trained to classify exercise types and detect repetition cycles, enabling accurate repetition counting across different users. The approach demonstrates the effectiveness of deep learning techniques in modeling complex motion patterns and improving automated exercise monitoring systems. However, the method relies heavily on large annotated datasets and accurate pose detection, and its performance may degrade when occlusions or tracking errors occur.

2.1.2 Viewpoint Invariant Exercise Repetition Counting Using Skeleton Data

This work [2] introduces a repetition counting method based on skeleton data extracted from human pose estimation. Instead of analyzing raw video frames, the system processes body joint coordinates to identify repetitive motion patterns. The proposed approach is designed to be viewpoint invariant, allowing accurate repetition counting even when the camera angle changes. By focusing on skeletal representations, the method reduces background noise and improves robustness across different recording environments. However, the approach is highly dependent on the accuracy of pose estimation algorithms, and errors in skeleton tracking may directly affect the repetition counting results.

2.1.3 Pūioio: On-Device Real-Time Smartphone-Based Automated Exercise Repetition Counting System

The Pūioio system [3] presents a lightweight repetition counting framework designed to run directly on smartphones. Using camera based pose estimation and motion state detection, the system identifies exercise movements and counts repetitions in real time. The architecture is optimized for mobile devices, making it suitable for consumer fitness applications and home workout monitoring. This approach eliminates the need for additional wearable sensors or specialized hardware, improving accessibility for users. However, system performance may decrease under challenging lighting conditions, rapid movements, or cluttered backgrounds that affect pose detection accuracy.

2.1.4 Recognizing Exercises and Counting Repetitions in Real Time

This study [4] presents a real time system capable of recognizing exercise types while simultaneously counting repetitions. The proposed approach integrates human pose estimation with machine learning classification models to analyze body movements captured from video input. By detecting repetitive motion cycles and identifying exercise types, the system provides real time feedback during workout sessions. Such systems are useful for intelligent coaching platforms and rehabilitation monitoring applications. However, the system performs best in controlled environments and may struggle with irregular repetition speeds or variations in individual movement patterns.

2.1.5 Workout Type Recognition and Repetition Counting with Wearable Sensors

This paper [5] investigates the use of wearable inertial measurement unit (IMU) sensors for recognizing workout types and counting repetitions. Motion signals collected from body mounted sensors are analyzed to detect exercise patterns and repetition cycles. Since the system directly measures body movement, it achieves high accuracy and is less affected by lighting conditions or visual occlusions. Such sensor based systems are widely used in sports analytics and rehabilitation monitoring. However, the requirement of wearable devices introduces additional hardware dependency and may reduce user convenience compared to camera based approaches.

2.2 Summary and Gaps Identified

Table 2.1: Comparison of Existing Exercise Repetition Counting Methods

Paper	Method Used	Advantages	Limitations
Soro et al. [1]	Deep learning motion analysis	Accurate repetition counting for multiple exercises and captures temporal motion patterns.	Requires large datasets and high computation. Sensitive to pose errors.
Hsu et al. [2]	Skeleton-based pose estimation	Robust to camera viewpoint variations and reduces background noise.	Depends on accurate skeleton tracking. Errors affect counting.
Sinclair et al. [3]	Smartphone pose estimation	Real-time repetition counting on mobile devices without extra hardware.	Sensitive to lighting and fast movements.
Alatiah and Choi [4]	Pose estimation + ML classification	Recognizes exercise type and counts repetitions simultaneously in real time.	Struggles with irregular repetition speeds and uncontrolled environments.
Skawinski et al. [5]	Wearable IMU sensors	High accuracy independent of lighting and camera positioning.	Requires wearable hardware and correct sensor placement.

2.2.1 Research Gap

- Most existing repetition counting systems assume consistent exercise patterns and struggle to handle variations in movement speed, body proportions, and execution styles across different users.
- Many vision-based approaches rely heavily on accurate pose estimation. Errors in joint detection or occlusions can significantly reduce the accuracy of repetition counting.
- Several studies focus primarily on counting repetitions but do not evaluate the correctness or quality of exercise movements.

- Camera based systems are sensitive to environmental conditions such as lighting variations, background clutter, and camera positioning.
- Sensor-based approaches require wearable hardware such as IMU sensors, which may reduce user convenience and increase system complexity.
- Many existing solutions lack integrated real time feedback mechanisms that can guide users and improve exercise performance during workouts.

2.3 Summary of Chapter

From the reviewed literature, it is evident that significant progress has been made in the development of automated systems for exercise recognition and repetition counting. Deep learning and pose estimation techniques have improved the ability to detect complex human movements and analyze repetitive exercise patterns. However, several limitations remain in existing approaches. Many vision based systems rely heavily on accurate pose detection and are sensitive to environmental conditions such as lighting variations and occlusions. Similarly, sensor based methods require wearable devices, which may reduce user convenience and increase system complexity.

Furthermore, many existing solutions focus primarily on repetition counting without evaluating exercise quality or providing real time feedback to users. Variations in individual movement patterns and exercise speed also present challenges for reliable performance analysis. These limitations highlight the need for more robust and user friendly systems that can accurately analyze human movements in real world environments while providing meaningful feedback to improve exercise performance.

Chapter 3

System Design

This chapter describes the design and implementation structure of the proposed posture evaluation system. It explains the overall system architecture, major components, module organization, technologies used, and project development plan.

3.1 Overall Architecture

The proposed system, CORE-ALIGN, is designed as a real time video based posture evaluation platform capable of analyzing exercise movements and providing corrective feedback. The system integrates a web based user interface, a backend motion analysis engine, and a cloud database to process workout sessions and evaluate user posture. The overall architecture consists of three main components: the frontend, backend, and database, as illustrated in Figure 3.1.

The frontend is implemented as a React-based web application [6] that manages user interaction with the system. It provides functionalities such as user authentication, workout session management, and performance visualization through the dashboard interface. During a workout session, the frontend captures video streams from both the user and the trainer reference video. Pose estimation and gesture detection are performed using MediaPipe [7] to extract skeletal landmarks and enable gesture based session control. The extracted landmark coordinates are transformed into joint angle representations, which describe body movement over time. These angle time signals are then transmitted to the backend through WebSocket communication for further processing.

The backend performs the core motion analysis operations and is implemented using FastAPI [8]. It receives joint angle data from the frontend and processes the signals through multiple motion analysis stages. These stages include signal filtering, temporal interpolation, primary joint-angle detection, and repetition segmentation. The segmented repetitions are then compared with trainer reference movements to evaluate posture accuracy. Based on this analysis, the backend generates corrective feedback that is transmitted back to the frontend for visualization.

The system uses Firebase Firestore [9] as the database for storing application data. The database maintains user information, workout session records, repetition statistics, and feedback results generated during exercise analysis. These records enable users to review their workout history and track performance progress over time.

The overall workflow of the system begins with user authentication and workout initiation through the frontend interface. Video frames are processed to extract pose landmarks and compute joint-angle signals, which are transmitted to the backend for motion analysis. The backend evaluates the exercise repetitions and generates performance metrics and feedback. The results are then displayed to the user through the frontend dashboard and stored in the database for future reference.

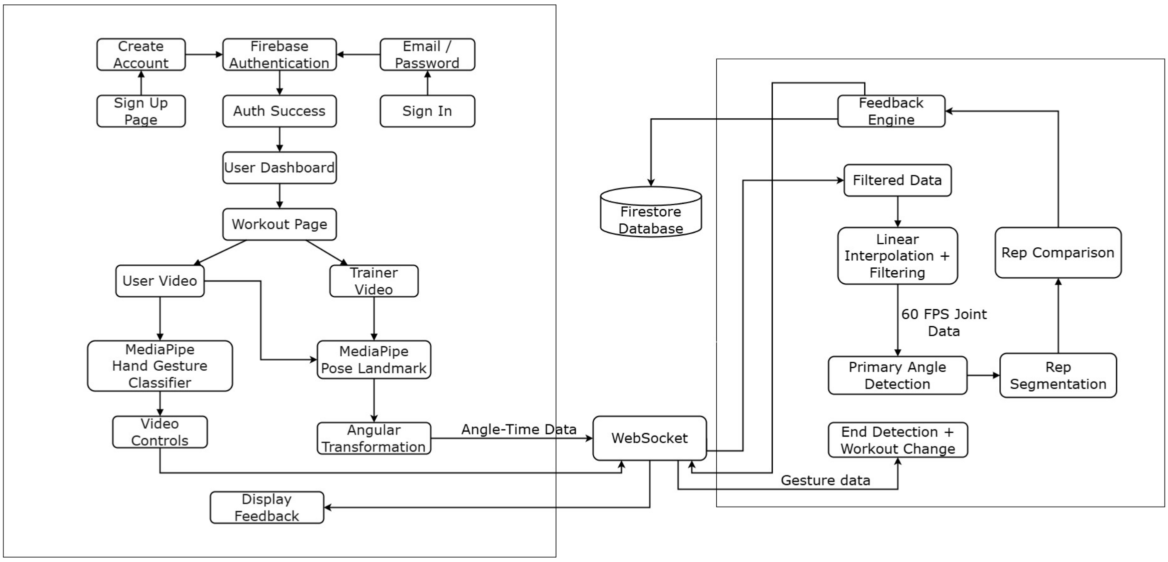


Figure 3.1: Overall Architecture of the CORE-ALIGN system.

3.2 Component Design

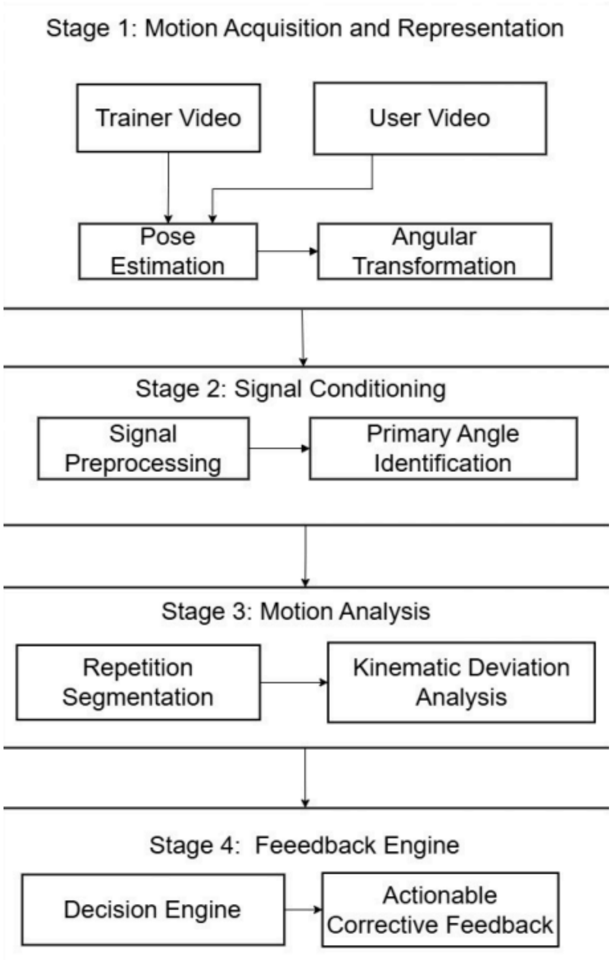


Figure 3.2: Motion analysis pipeline for posture evaluation.

The proposed posture evaluation framework is organized into multiple modules that together implement the motion analysis pipeline and supporting system functionality. These modules transform raw exercise videos into structured motion signals, perform repetition analysis, and generate corrective feedback.

The core components include motion acquisition and representation, signal conditioning, motion analysis, and the feedback engine. In addition to these processing modules, the system also includes supporting modules for gesture-based interaction and cloud data management. Gesture detection enables hands-free workout control, while Firebase services manage user authentication and storage of workout data.

3.2.1 Motion Acquisition and Representation

This component captures human motion from video and converts it into a structured kinematic representation suitable for computational analysis. The system processes two video inputs: a live webcam stream capturing the user performing an exercise and a pre-recorded trainer reference video.

Human pose extraction is performed using the MediaPipe Pose framework [7], which provides real-time skeletal tracking from RGB video streams. MediaPipe detects a set of body landmarks corresponding to major joints and returns their three-dimensional spatial coordinates.

Each frame is processed to extract body joint coordinates. Let

$$J_t = \{(x_i, y_i, z_i)\}_{i=1}^{N} \tag{3.1}$$

represent the detected joint set at time t, where N denotes the number of tracked joints. These joint trajectories describe the spatial configuration of the human body across time. To obtain a biomechanically meaningful representation of motion, the joint coordinates are transformed into angular trajectories. For three joints a, b, and c, where b is the vertex, the joint angle θ is defined as

$$\theta = \cos^{-1}\left(\frac{(a-b)\cdot(c-b)}{\|a-b\|\|c-b\|}\right) \tag{3.2}$$

The resulting angle–time sequences represent exercise motion as coordinated rotations of body segments and serve as the primary input for subsequent motion analysis.

In addition to pose estimation, the MediaPipe framework is also used for gesture detection to enable hands-free control of workout sessions. Detected hand landmarks are analyzed to recognize predefined gestures that control exercise playback and interaction within the system.

3.2.2 Signal Conditioning

Joint-angle trajectories obtained from pose estimation may contain noise and irregular sampling artifacts caused by prediction uncertainty, motion blur, or frame rate variations. The signal conditioning component stabilizes these trajectories before motion analysis.

Noise reduction is performed using recursive temporal smoothing

$$\tilde{\theta}_j(t) = \alpha\theta_j(t) + (1-\alpha)\tilde{\theta}_j(t-1) \tag{3.3}$$

where α controls the smoothing strength. Temporal normalization is then achieved by interpolating the signals onto a fixed time grid to ensure consistent sampling across recordings captured under different conditions.

To isolate motion defining joints, the system identifies *primary angles* based on motion variability. The standard deviation of each angular trajectory is computed and joints are ranked in descending order. A threshold derived from the largest deviation gap separates highly active joints from relatively static ones. Only these selected primary angles are retained for further motion analysis.

3.2.3 Motion Analysis

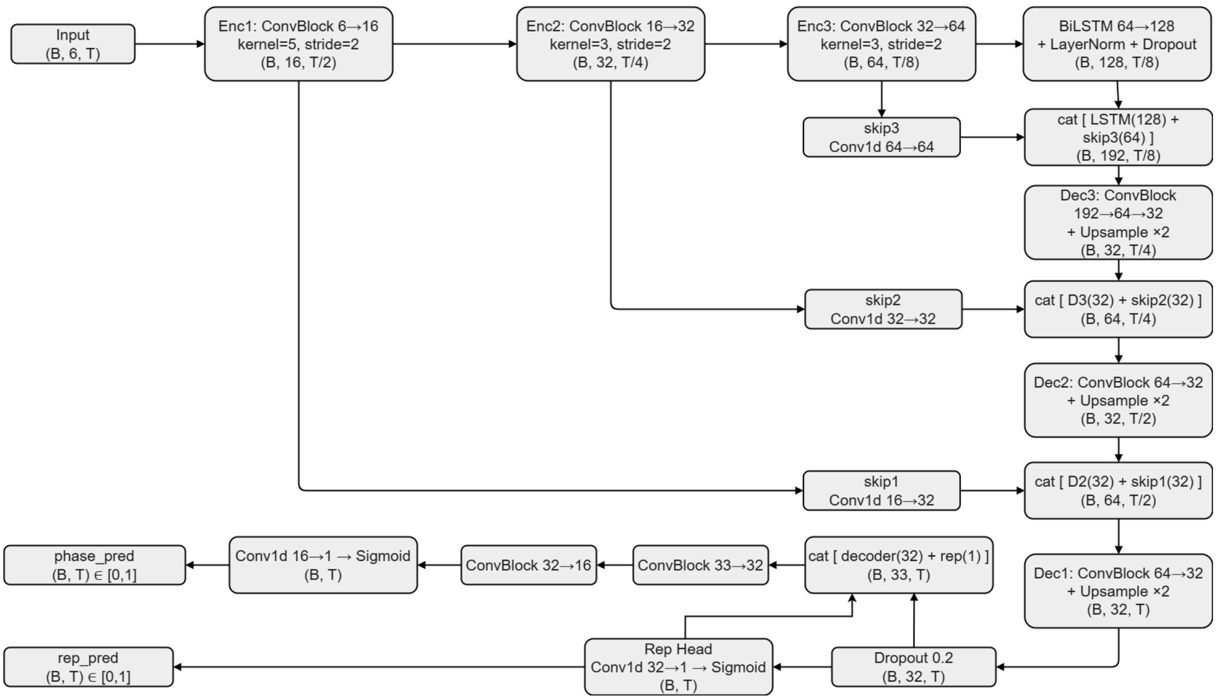


Figure 3.3: Architecture of U-net BiLSTM Model.

The motion analysis component detects exercise repetitions and analyzes movement patterns from the conditioned joint angle signals. Repetition segmentation is performed using a supervised neural network trained on temporal joint angle sequences as given in Figure 3.3.

The backend processing pipeline is implemented using the FastAPI framework, which provides an asynchronous interface for handling real time motion analysis tasks. Joint angle signals extracted from the frontend are transmitted to the backend through WebSocket communication. This enables continuous bidirectional data exchange between the frontend and backend, allowing the system to process motion data incrementally during workout sessions.

The network receives joint angle sequences organized as a multichannel time series

$$X \in \mathbb{R}^{C \times T} \tag{3.4}$$

where C denotes the number of selected primary joint angles and T represents the temporal length of the input sequence.

Encoder Blocks

The encoder stage extracts hierarchical motion features from the input sequence using a series of one dimensional convolutional blocks. Each block consists of a convolution layer followed by batch normalization and a rectified linear activation function. Temporal downsampling is performed using stride operations, allowing the network to progressively capture motion patterns across larger temporal contexts.

BiLSTM Bottleneck

The encoded feature representation is processed by a Bidirectional Long Short Term Memory (BiLSTM) layer [10]. The BiLSTM analyzes the sequence in both forward and backward directions, enabling the model to capture long range temporal dependencies and contextual relationships between motion segments. Layer normalization and dropout are applied to improve training stability.

Decoder Blocks

The decoder reconstructs frame level predictions by progressively restoring temporal resolution through upsampling layers. Skip connections between encoder and decoder stages preserve fine-grained motion information, allowing the decoder to combine high-level temporal context with low-level motion features.

Output Layers

The network produces two complementary outputs. The first output predicts the repetition activity signal

$$y_r(t) \in [0, 1] \tag{3.5}$$

which indicates whether each frame belongs to an active repetition segment. The second output predicts the repetition phase

$$y_p(t) \in [0, 1] \tag{3.6}$$

which represents the normalized progression of motion within a repetition cycle.
These outputs allow the system to segment continuous motion into structured exercise repetitions.

3.2.4 Feedback Engine

The feedback engine evaluates the quality of each detected repetition by comparing the user motion trajectory with a trainer reference movement. Because repetition durations may vary, both trajectories are temporally normalized before comparison.
Let $x_u(t)$ and $x_r(t)$ represent the aligned user and reference joint angle signals. Structural similarity between the trajectories is evaluated using error-based metrics including Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and maximum deviation.
The range of motion for each repetition is defined as

$$ROM = \max(x) - \min(x) \tag{3.7}$$

and execution speed is estimated using the ratio between user and trainer repetition durations.
Each metric is mapped to a normalized performance score representing posture accuracy, movement completion, and execution timing. These scores are combined into a weighted quality index that determines the overall performance of the exercise repetition. Based on this evaluation, the system generates corrective feedback to guide the user toward improved exercise execution.

3.2.5 Gesture Interaction Module

The system incorporates a gesture interaction module to enable hands free control of workout sessions. Gesture detection is implemented using the MediaPipe Hands framework [7], which detects hand landmarks from the webcam video stream and identifies predefined gestures based on landmark configurations.

Three primary gestures are used to control workout navigation. A single raised index finger gesture is mapped to the play command, which starts or resumes the workout session. A two finger gesture is mapped to the pause command, allowing the user to temporarily stop the exercise. The next exercise command is triggered by a combined gesture sequence consisting of a thumbs up followed by a thumbs down gesture. This gesture control allows users to interact with the workout interface without manual input, improving usability during active exercise sessions.

3.2.6 Cloud Data Management Module

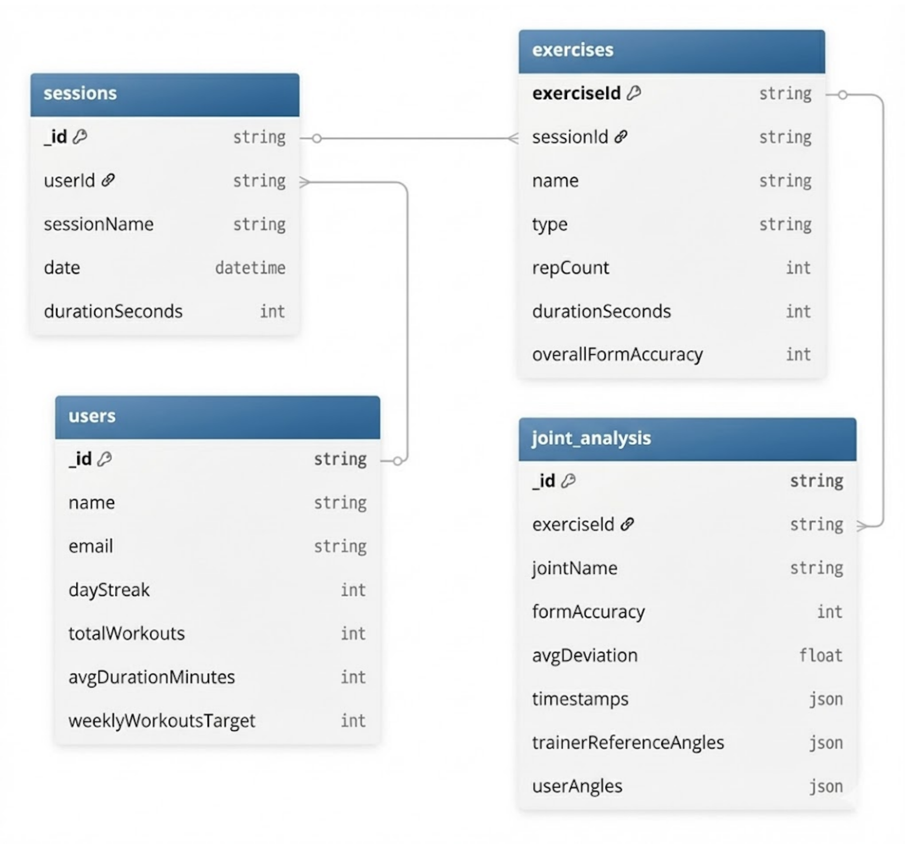


Figure 3.4: Datbase Schema

User management and data storage in the proposed system are implemented using Firebase cloud services [9]. Authentication is handled through Firebase Authentication, which provides secure user registration, login, and session management using both email based credentials and third party providers.

Workout related data is stored in Cloud Firestore, a scalable NoSQL database designed for real time applications. The database follows a hierarchical document based schema, as illustrated in Fig. 3.4, where each user is associated with multiple workout sessions. Each session contains a collection of exercises, and each exercise further stores detailed joint level analysis results.

The *users* collection maintains user specific information such as profile details and activity statistics. The *sessions* collection stores workout session metadata including session name, date, and duration. Each session contains multiple entries in the *exercises* collection, which record exercise type, repetition count, duration, and overall form accuracy. For fine grained analysis, the *joint_analysis* collection stores joint wise performance metrics, including deviation values, timestamps, and both user and trainer reference angle data.

This schema enables efficient organization and retrieval of workout data, supports real-time synchronization with the frontend dashboard, and allows users to track historical performance and analyze improvements over time.

3.3 Module Division

3.3.1 Module 1: User Interaction and Frontend Interface

Module 1 serves as the primary interaction layer between the user and the posture correction system. It manages user authentication and session handling, provides the workout execution interface, enables gesture-controlled interaction during exercise sessions, visualizes performance analytics, and presents detailed exercise-level feedback to the user. The frontend is implemented using a React-based architecture [6], which enables modular user interface components and dynamic state management for rendering session data and analytics in real time. Secure user login and registration are handled through Firebase Authentication [9], ensuring session persistence, scalable multi-user support, and proper isolation of user-specific data. Structured workout data such as session history, repetition counts, timestamps, and feedback metrics are stored using Cloud Firestore, where records are indexed per authenticated user for efficient retrieval. Additionally, MediaPipe gesture detection [7] is integrated into the frontend to enable hands free control of workout sessions through real-time gesture recognition.

3.3.2 Module 2: Pose Extraction and Kinematic Signal Processing

Module 2 is responsible for real time pose extraction, angular transformation, and signal conditioning to convert raw user and trainer motion data into stable, analysis-ready kinematic signals. The system utilizes MediaPipe Pose [7] for real time skeletal tracking, extracting 33 three dimensional landmark coordinates (x, y, z) representing major body joints. Pose estimation is performed independently on both the user and trainer video streams, enabling frame wise landmark detection and maintaining temporal continuity of motion data. These landmark coordinates are transformed into biomechanically meaningful joint angles, forming the primary representation of posture for subsequent analysis. To address noise introduced by landmark detection errors and camera instability, a Hybrid Incremental Filtering approach is applied. The filtering process operates incrementally without reprocessing historical data, maintaining separate filter instances for raw angle data and interpolated signals to support real time performance with minimal memory overhead. After filtering, linear interpolation is applied to normalize the temporal resolution of the signals to a fixed rate of 60 frames per second, compensating for irregular timestamps caused by variable camera frame rates. This resampling ensures consistent temporal alignment between user and trainer motion data, enabling fair comparison and reliable statistical analysis. For real time processing, joint angle data from each frame, along with corresponding trainer angles and gesture information, are organized according to timestamps and transmitted to the backend in sequential batches via WebSockets. Each batch represents approximately one second of motion data, ensuring continuous communication, low latency, controlled memory usage, and efficient real time posture evaluation.

3.3.3 Module 3: Repetition Detection and Motion Segmentation

Module 3 converts processed joint angle signals into structured exercise repetitions by identifying dominant movement patterns and segmenting motion cycles. The system first performs statistical analysis on all joint angle signals to determine the joints that contribute most significantly to the exercise motion. This is achieved by computing the standard deviation of each joint angle over time, where higher variability indicates stronger participation in the movement. The joint angles are then ranked in descending order of motion intensity, and a maximum difference method is applied to identify the natural separation between dominant and non-dominant joints. The selected joints are treated as primary angles representing the key biomechanical drivers of the exercise. If the maximum variation across all joints is below a predefined threshold, the system interprets the motion as a rest state rather than a valid exercise movement.

Once the primary angles are identified, a temporally aware deep learning model based on a U-Net [11] and BiLSTM [10] architecture is used to classify each frame into motion phases such as rest, upward movement, or downward movement. By analyzing transitions between these phases, the system accurately detects repetition boundaries and segments the motion into individual exercise cycles. Each repetition is assigned precise start and end timestamps, allowing the system to compute repetition counts and prepare segmented motion data for further comparison with the trainer's reference movement. This structured segmentation enables consistent analysis across exercises performed at different speeds or styles.

3.3.4 Module 4: Posture Comparison and Feedback Generation

Module 4 performs posture evaluation by comparing the segmented user repetitions with the trainer's reference motion to detect deviations in exercise execution. For each repetition identified in the previous stage, the system aligns the user's joint angle trajectories with the corresponding trainer motion. This alignment enables a frame by frame comparison of joint behavior, allowing the system to quantify differences in movement patterns, joint angles, and execution timing. Deviation metrics are calculated to identify posture errors such as insufficient range of motion, incorrect joint alignment, or inconsistent movement patterns during an exercise.

Based on these deviation measurements, the system generates meaningful corrective feedback for the user. Joint specific errors are translated into interpretable guidance that can be displayed on the user dashboard during or after the workout session. The module also produces performance metrics including repetition count, motion consistency, and deviation summaries for each exercise. These results are transmitted back to the frontend interface where they are visualized through analytics dashboards and feedback panels. By converting raw motion comparisons into actionable insights, this module completes the pipeline and enables the system to function as an intelligent virtual posture coach.

3.4 Tools and Technologies

3.4.1 Software Requirements

- **Frontend Framework:** React
- **Backend Framework:** FastAPI (Python)
- **Real-Time Communication:** WebSocket
- **Video Processing:** MediaPipe Pose Landmark, MediaPipe Hand Gesture Classifier
- **Database:** Firebase

3.4.2 Hardware Requirements

- **Processor:** Intel i5 (10th Gen or above) / AMD Ryzen 5 or better
- **RAM:** Minimum 8 GB (Recommended: 16 GB for smooth rendering)
- **GPU:** 4 GB VRAM or above
- **Webcam:** Required (for user pose capture)
- **Operating System:** Windows 10/11, macOS, or modern Linux
- **Browser:** Latest version of Chrome, Edge, or Firefox

3.5 Work Breakdown and Responsibilities

3.5.1 Aaron Joseph: User Dashboard & Gesture Interaction

- Design and develop the React-based user interface.
- Implement user authentication and session management using Firebase.
- Build the Workout Dashboard and analytics visualization.

- Enable gesture-controlled workout navigation.
  - Play / Pause / Next Exercise
- Display real-time feedback received from backend modules.

3.5.2 Bijay Krishna D: Pose Estimation & Motion Processing

- Extract 33 body landmarks using MediaPipe Pose.
- Convert landmark coordinates into joint angles.
- Perform signal preprocessing.
  - Noise filtering
  - Interpolation
  - Temporal normalization
- Handle real-time video processing.
- Structure motion data and send batches via WebSockets.

3.5.3 Abin J Arackal: Repetition Counting & Movement Analysis

- Identify primary moving joints dynamically.
- Normalize exercise cycles.
- Prepare and train the dataset.
- Implement an intelligent repetition segmentation model.
- Segment each repetition for further comparison.

3.5.4 Angelo Varghese: Comparison & Feedback System

- Generate ideal reference movement from trainer data.
- Compare user repetitions with the reference movement.
- Measure deviation in repetitions.

- Detect form mistakes automatically.
- Generate real-time corrective feedback.
- Send feedback to the dashboard for display.

3.6 Key Deliverables

The primary outcome of this project is CORE-ALIGN, a real-time video-based posture evaluation system capable of analyzing exercise movements and providing corrective feedback during workout sessions. The system includes a web-based workout platform that enables users to perform exercises while receiving real time feedback and visualizing performance metrics through a dashboard interface. Gesture interaction is incorporated to support hands free workout control.

The implementation also includes a motion analysis pipeline that converts exercise videos into joint angle signals and performs repetition segmentation and movement analysis. An intelligent repetition analysis model is developed to detect exercise repetitions, along with a feedback mechanism that evaluates posture accuracy, range of motion, and execution speed to generate corrective guidance. In addition, a data management system using Firebase Authentication and Firestore is integrated to manage user accounts and store workout session data.

3.7 Project Timeline

The development of the proposed posture evaluation system was carried out over a period of approximately nine months, from June to March. The project was divided into several phases including research and planning, development of individual system modules, and final system integration. The modules were implemented sequentially, beginning with the user dashboard and gesture interface, followed by pose estimation and angular transformation, primary angle detection and repetition segmentation, and the comparison and feedback module. The final stage involved integrating all modules into a complete working system. Figure 3.5 illustrates the timeline and overlap of these development activities across the project duration.

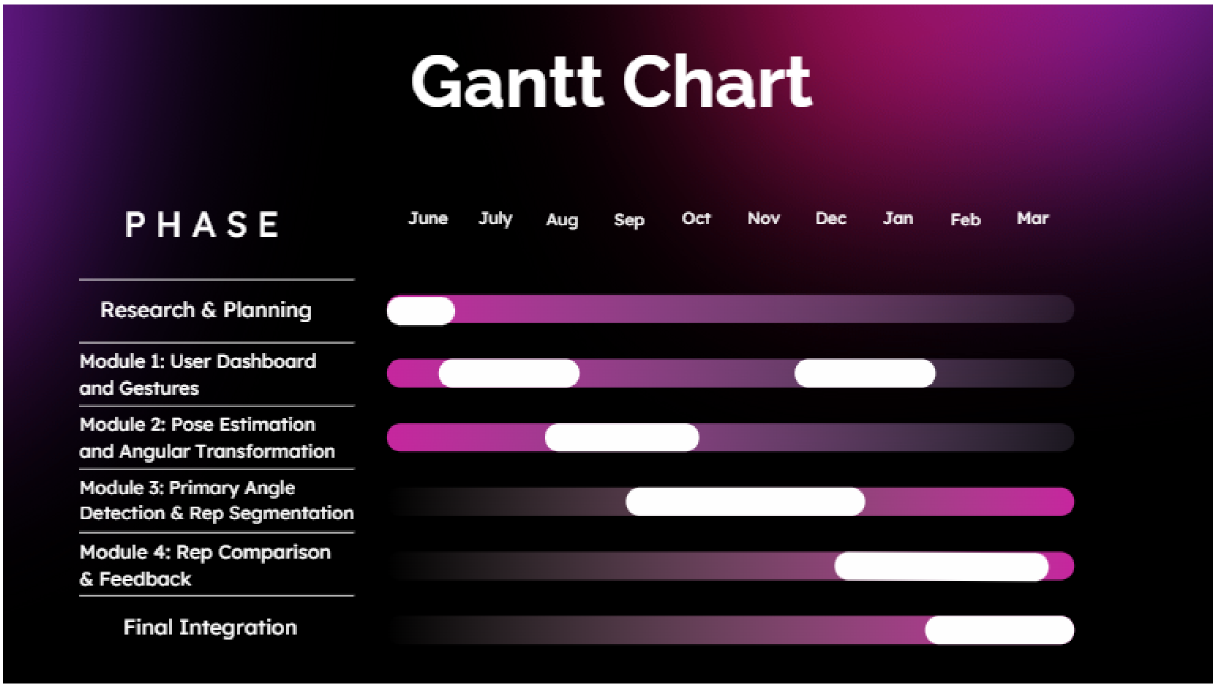


Figure 3.5: Gantt Chart

3.8 Summary of Chapter

This chapter presented the design and architecture of the proposed posture evaluation system. The system components and their interactions within the motion analysis pipeline were described, including motion acquisition, signal conditioning, motion analysis, and the feedback engine. Additional supporting modules such as gesture-based interaction and cloud-based data management were also discussed. The chapter further outlined the technologies and frameworks used, the division of responsibilities among team members, key project outputs, and the development timeline. Together, these elements define the overall structure and implementation approach of the system.

Chapter 4

Results and Discussions

This chapter presents the results obtained from the implementation of the proposed posture evaluation system and analyzes its performance under various conditions. The evaluation includes both quantitative analysis of the motion analysis model and qualitative assessment of the system behavior during real-time operation.

The performance of the model is examined using metrics such as validation accuracy, loss curves, and confusion matrices to evaluate its ability to detect repetition patterns. In addition, the system-level performance is demonstrated through implementation results, including the user interface, real-time feedback generation, and gesture-based interaction. The results are further discussed to highlight the strengths, limitations, and practical applicability of the proposed system.

4.1 Dataset Description

The dataset used for training and evaluation was constructed manually using a combination of self-recorded exercise videos and publicly available trainer videos sourced from online platforms. This approach was adopted to capture realistic variations in exercise execution across different individuals and environments. The dataset creation process involved modifying the project pipeline to extract pose-based motion features and store them as structured CSV files.

All videos were processed using the pose estimation module to extract joint-angle representations, which were then stored as time-series data. Dataset generation and preprocessing were performed on systems equipped with NVIDIA GTX 1650 and RTX 3060 GPUs to ensure efficient processing of video sequences.

The final dataset consists of a total of 792 CSV files generated from 66 different exercise instances. Each exercise sample was augmented to produce 12 variations, resulting in improved diversity in motion patterns. The dataset was split into training and validation sets based on different individuals to ensure generalization across users. The training set consists of data from 52 individuals (624 files), while the validation set consists of data from 14 individuals (168 files).

The exercises in the dataset include a variety of lower-body and full-body movements such as squats, lunges, deadlifts, and dynamic compound exercises. The use of both trainer and user-generated videos introduces variability in execution style, speed, and posture, which enhances the robustness of the model. However, since the dataset was manually created and limited in size, it may not fully capture all real-world variations, which can impact generalization performance.

4.2 Experimental Setup

The training process was performed on the generated dataset using a sliding window approach to capture temporal motion patterns. Joint-angle sequences were segmented into fixed-length windows of 480 frames with a stride of 48 frames. This resulted in a total of 28,668 training samples and 6,080 validation samples, each represented as a multichannel time series with 6 joint-angle features.

The model was trained using a batch size of 64 for 120 epochs. An initial learning rate of 3×10^{-3} was used. The training was performed in a staged manner, where the repetition detection head was trained first, followed by joint optimization with the phase prediction head.

To improve robustness, a focal loss function with a focusing parameter of $\gamma = 2.0$ was used for repetition classification, along with label smoothing. Additional loss components were incorporated to enforce temporal consistency, including phase prediction loss, repetition count regularization, repetition length constraints, and phase monotonicity. These constraints help the model learn structured repetition patterns and improve segmentation accuracy.

4.3 Model Results

The performance of the proposed model was evaluated using frame-level classification metrics, confusion matrices, and training curves. The best-performing model achieved

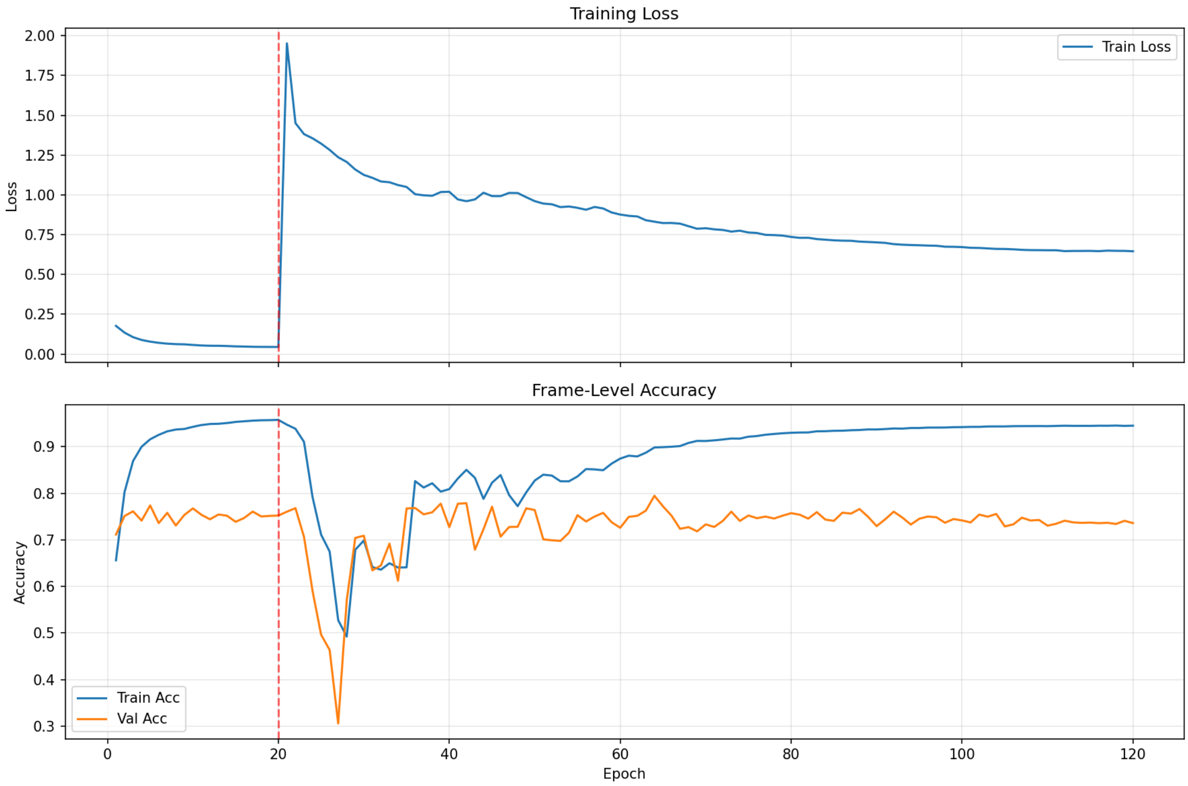


Figure 4.1: Validation Curve

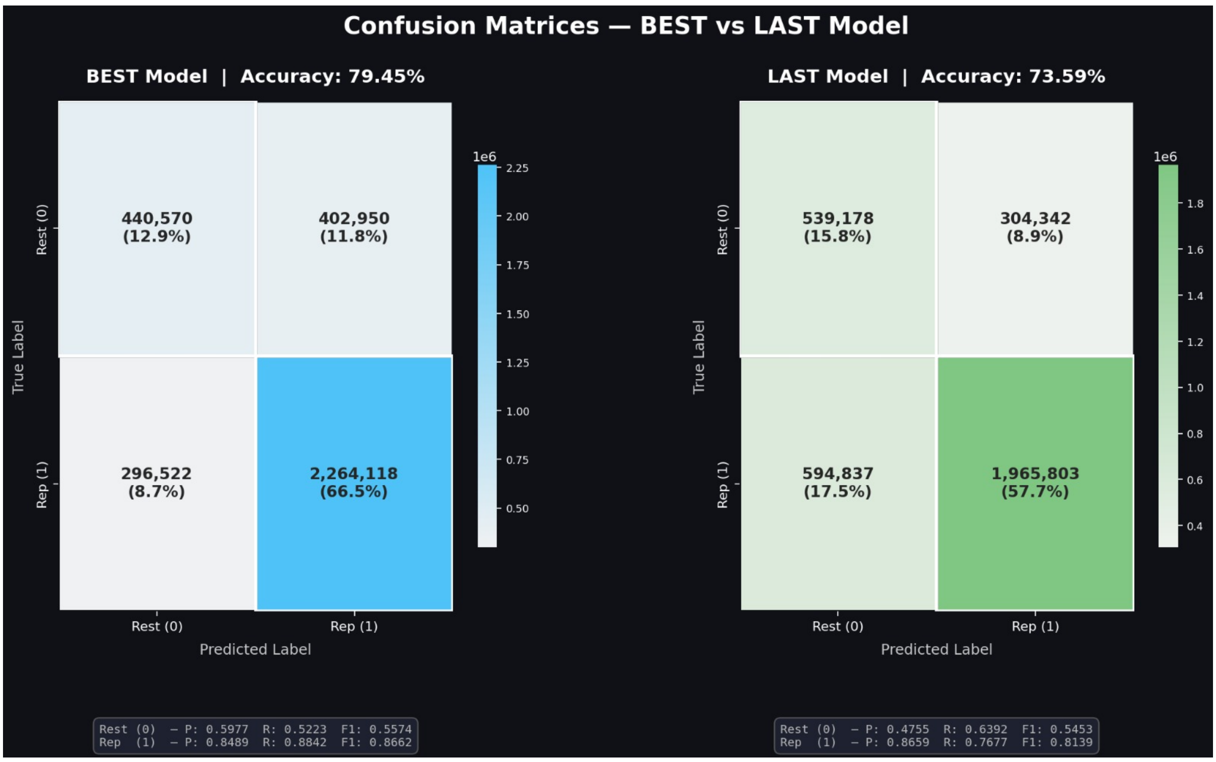


Figure 4.2: Confusion Matrix

an overall validation accuracy of approximately 79.45%, while the final trained model achieved an accuracy of 73.58%. These results indicate that the model is capable of learning meaningful temporal motion patterns despite the challenges associated with pose estimation noise and dataset limitations.

The confusion matrix of the best model shows strong performance in detecting repetition frames, with a high recall of 88.42% for the Rep class. However, the model exhibits comparatively lower performance in identifying Rest states, with recall around 52.23%, indicating confusion near transition boundaries between rest and active motion. This behavior is expected in temporal segmentation tasks, where the distinction between motion phases is often gradual rather than discrete.

The final model demonstrates slightly lower performance compared to the best model, with increased misclassification between Rest and Rep states. In particular, the confusion matrix indicates a higher number of false positives for the Rep class, suggesting that the model tends to over-predict active motion in ambiguous regions. Despite this, the model maintains a strong ability to detect repetition segments, as reflected by the relatively high F1-score of 0.8139 for the Rep class.

The training curves illustrate the learning behavior of the model across epochs. The loss initially decreases steadily, indicating effective optimization during the early training phase. A noticeable transition is observed at the staged training point, after which the model stabilizes and continues to improve gradually. The training accuracy reaches above 90%, while validation accuracy stabilizes around 73–79%, indicating a moderate generalization gap. This gap can be attributed to dataset limitations and variability in real-world motion patterns.

Overall, the results demonstrate that the proposed model achieves reliable performance in repetition detection under real-time conditions. While some misclassification occurs near motion boundaries, the model effectively captures the overall structure of exercise repetitions. Further improvements in dataset size and model regularization could enhance classification accuracy and reduce boundary ambiguity.

4.4 System Implementation Results

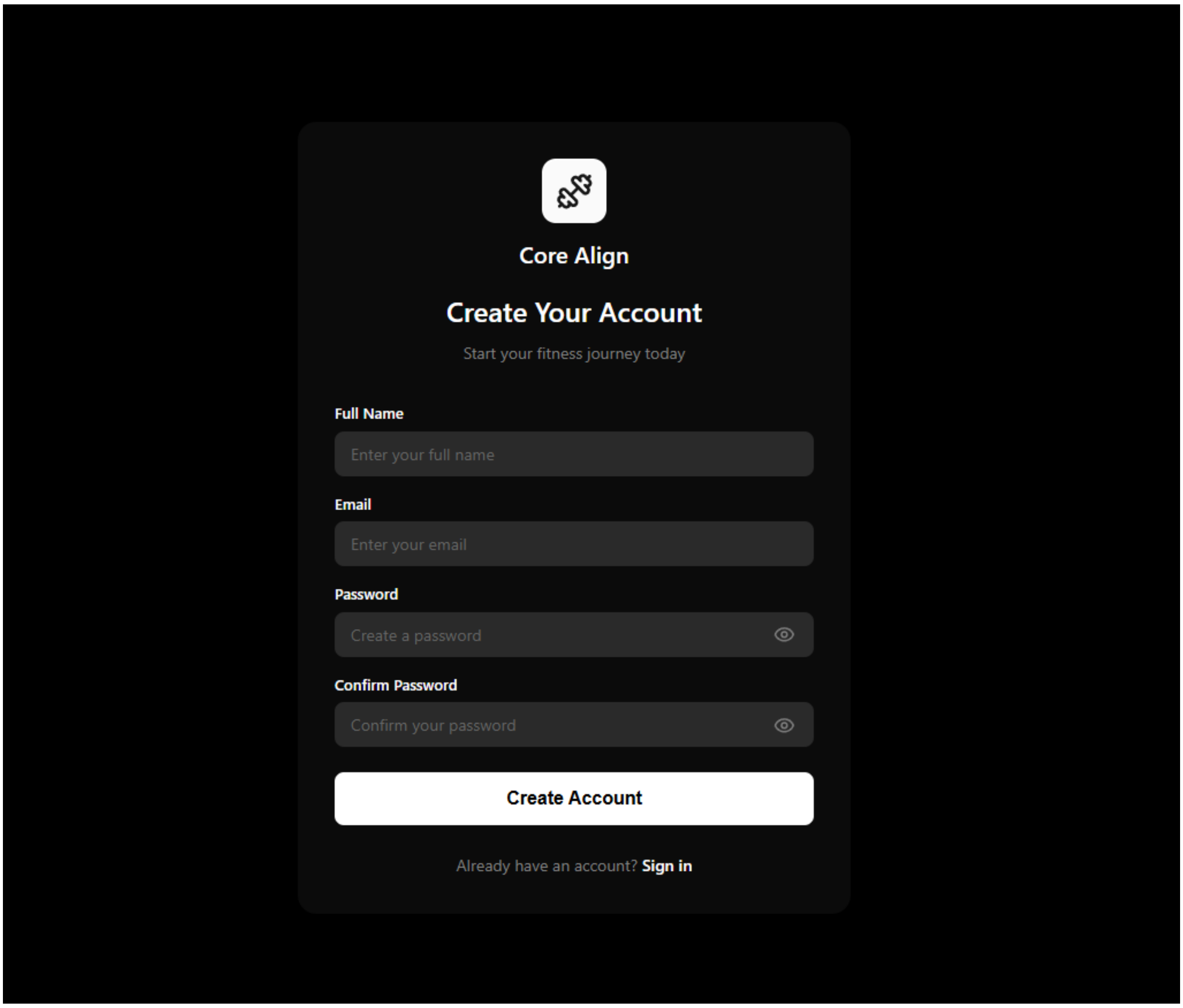


Figure 4.3: System Workflow Overview

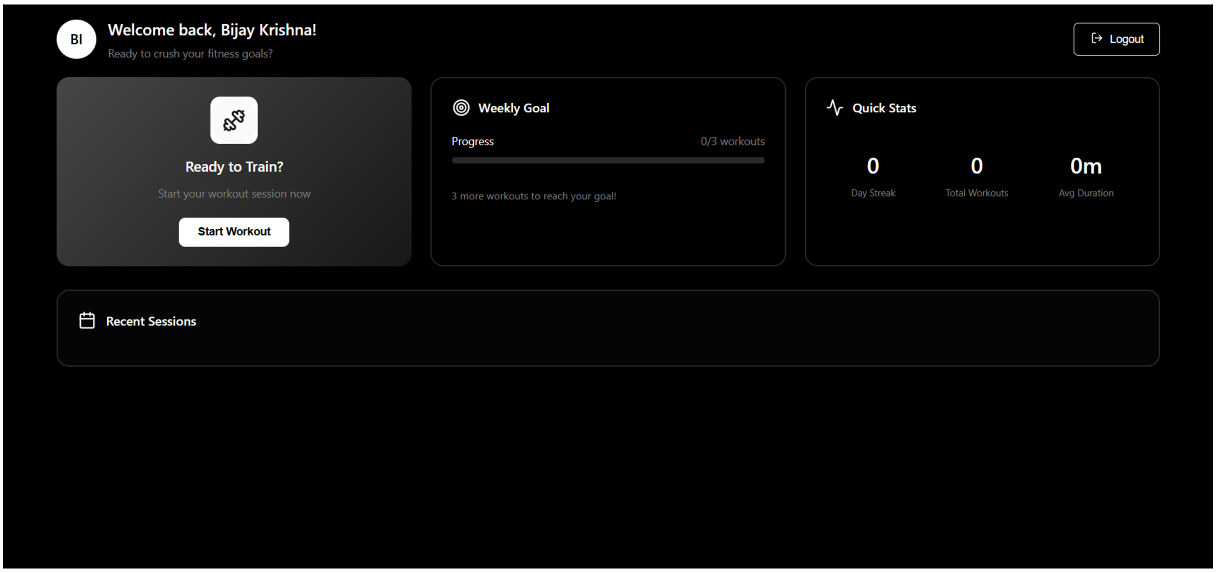


Figure 4.4: User Dashboard

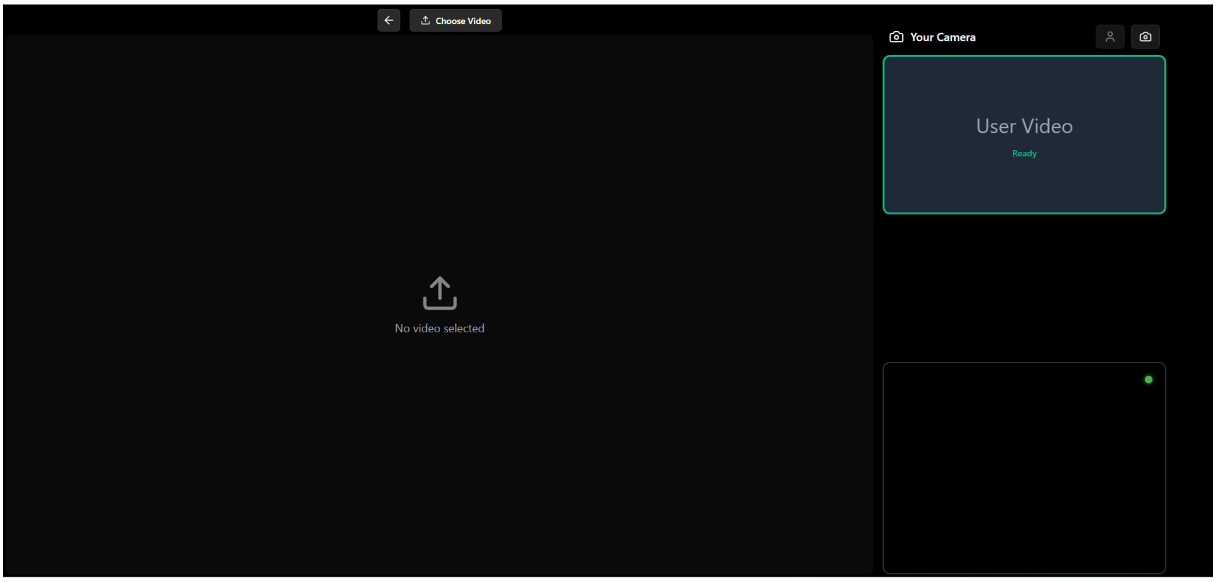


Figure 4.5: Workout Page - Initial View

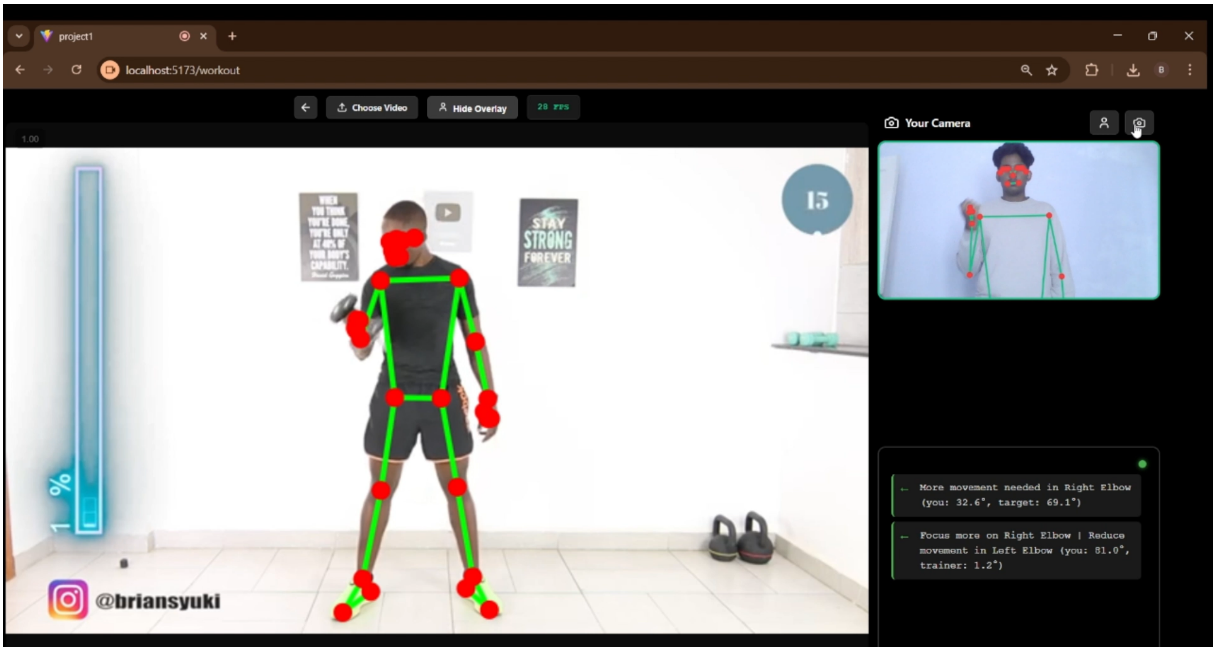


Figure 4.6: Workout Page - Active Feedback

The implemented system was evaluated based on its real-time performance, usability, and integration of different modules. The system successfully provides a complete workflow from user authentication to real-time posture feedback during workout sessions.

The user authentication module functioned reliably, supporting both email-based login and Google sign-in through Firebase Authentication. Users were able to access the system without significant delay, and session management was handled efficiently using cloud-based storage.

The workout interface demonstrated effective real-time performance, allowing users to perform exercises using either live webcam input or uploaded video files. The system was able to process motion data continuously and display feedback dynamically during workout execution. The feedback provided includes posture correction, repetition status, and movement quality indicators, enabling users to adjust their performance in real time.

The integration of gesture-based interaction enhanced the usability of the system by enabling hands-free control. Users were able to perform actions such as play, pause, and switching exercises using predefined gestures. The gesture system was highly responsive and worked effectively for single-hand inputs. However, it showed limitations when multiple hands were present in the frame, which occasionally resulted in incorrect gesture detection.

The real-time processing pipeline achieved stable performance, with smooth frame handling and continuous feedback generation. The system maintained responsiveness during most sessions, although minor interruptions were observed due to occasional WebSocket connectivity issues. Despite this, the system was able to recover quickly and resume normal operation.

Overall, the system implementation results demonstrate that the proposed approach is capable of delivering a functional and interactive real-time posture evaluation system. The integration of multiple components, including motion analysis, gesture interaction, and cloud services, provides a seamless user experience suitable for practical fitness applications.

4.5 Discussion

The proposed system demonstrates effective real-time performance for posture evaluation using standard video input. The system achieved a stable processing rate of approximately 50-60 frames per second on a system equipped with an NVIDIA GTX 1650 GPU, ensuring smooth motion capture and analysis during workout sessions. Feedback generation is controlled through a predefined time interval, which can be adjusted based on system requirements to balance responsiveness and computational load.

The communication between the frontend and backend using WebSocket based transmission was generally reliable and did not result in data loss during operation. However, intermittent connectivity issues were observed in certain cases, which could temporarily interrupt continuous data streaming. Despite this, the system maintained overall functional stability during most sessions.

The gesture interaction module enabled fast and responsive control of workout navigation. Gesture recognition performed well for single-hand inputs, allowing efficient hands-free interaction. However, the system exhibited limitations when multiple hands were present in the frame, leading to incorrect gesture detection. This indicates the need for improved robustness in multi-hand scenarios.

The pose estimation component plays a critical role in system performance. It was observed that the model is sensitive to occlusions, which frequently resulted in inaccurate joint detection and incorrect angle computation. Additionally, the presence of multiple individuals in the frame caused ambiguity in pose estimation, leading to unreliable motion data. It was also observed that after events such as video pausing or timeline scrolling, the model required a brief stabilization period due to sudden changes in the trainer's position. During this short interval, joint detection was inconsistent; however, the system quickly re-established accurate tracking. These limitations directly affect downstream processes such as repetition segmentation and posture evaluation.

The dataset used for training consisted of 66 exercises, each containing approximately 2500 frames. To improve training diversity, data augmentation was applied to generate multiple variations, resulting in 12 sequences per exercise. The dataset includes exercises performed by different individuals, introducing variation in movement patterns. However, the relatively limited dataset size and controlled data collection conditions may restrict the generalization capability of the model in complex real-world environments.

The overall system performance is also influenced by dataset size and computational resources. With a larger and more diverse dataset, the model would be able to learn richer motion patterns and improve its generalization capability. Additionally, the use of more powerful hardware would enable more efficient processing and model optimization, which could further enhance repetition detection and feedback accuracy.

Overall, the system demonstrates strong performance in controlled environments with a single user and minimal occlusions. The real-time capability, combined with gesture-based interaction and web-based deployment, makes the system practical for home-based fitness applications. However, improvements in pose estimation robustness, dataset diversity, and multi-person handling are necessary to enhance reliability in more complex scenarios.

Chapter 5

Conclusions & Future Scope

5.1 Conclusion

This project presented CORE-ALIGN, a real-time video-based posture evaluation system designed to analyze exercise movements and provide corrective feedback during workout sessions. The system integrates computer vision, motion signal processing, and web technologies to enable accurate posture assessment using standard video input. By converting video data into joint-angle signals and analyzing motion patterns through a structured pipeline, the system effectively detects exercise repetitions and evaluates user performance based on posture accuracy, range of motion, and execution speed.

The implementation demonstrates the feasibility of real-time posture analysis without the need for wearable sensors or specialized hardware. The inclusion of gesture-based interaction improves usability, while cloud-based data management supports tracking of user performance over time. Overall, the system provides a scalable and accessible solution for improving exercise safety and effectiveness in home-based fitness environments.

5.2 Future Scope

Future work can focus on improving the robustness of pose estimation under varying lighting conditions and occlusions. The repetition analysis model can be enhanced using larger and more diverse datasets to support complex and dynamic exercises. The system can be optimized for deployment on mobile or edge devices to improve accessibility and performance. Additional features such as personalized feedback, adaptive coaching, and multi-user support can further enhance usability. Integration with wearable devices or advanced sensors can also be explored to improve accuracy.

References

[1] A. Soro, S. Brunner, and R. Wattenhofer, "Recognition and repetition counting for complex physical exercises using deep learning," in *Sensors*, vol. 19, no. 3. MDPI, 2019, p. 714.

[2] Y.-C. Hsu, C.-Y. Chen, and Y.-L. Lin, "Viewpoint-invariant exercise repetition counting using skeleton data," *IEEE Access*, vol. 11, pp. 12 345–12 356, 2023.

[3] A. Sinclair, K. Kautai, and S. R. Shahamiri, "Pūioio: On-device real-time smartphone-based automated exercise repetition counting system," *Applied Sciences*, vol. 13, no. 5, p. 3123, 2023.

[4] T. Alatiah and J. Choi, "Recognizing exercises and counting repetitions in real time," in *2020 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE, 2020, pp. 1–4.

[5] K. Skawinski, P. Biecek, and J. M. Jasiński, "Workout type recognition and repetition counting with wearable sensors," *Sensors*, vol. 19, no. 20, p. 4459, 2019.

[6] Meta Platforms, Inc., "React: A javascript library for building user interfaces," https://reactjs.org/, 2026, accessed: 2026-03-17.

[7] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee *et al.*, "Mediapipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019.

[8] S. Ramírez, "Fastapi framework, high performance, easy to learn, fast to code, ready for production," https://fastapi.tiangolo.com/, 2026, accessed: 2026-03-17.

[9] Google LLC, "Firebase: An app development platform," https://firebase.google.com/, 2026, accessed: 2026-03-17.

[10] A. Graves and J. Schmidhuber, "Framewise phoneme classification with bidirectional lstm and other neural network architectures," *Neural Networks*, vol. 18, no. 5-6, pp. 602–610, 2005.

[11] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015*. Springer, 2015, pp. 234–241.

Appendix A: Presentation

CORE-ALIGN: Real-Time Video-Based Posture Evaluation System

100% Evaluation Presentation

Guided By
Ms. Dincy Paul

Group 11
Aaron Joseph (U2203004)
Abin J Arackal (U2203013)
Angelo Varghese (U2203043)
Bijay Krishna D (U2203068)

Table of Contents

Problem Definition

- The Posture Problem: Maintaining proper form during home workouts is difficult without immediate professional feedback.
- The Consequences: This lack of guidance increases the risk of injury and reduces overall training effectiveness.
- The Technology Gap: Current virtual systems cannot perform real-time posture comparisons between a user and a trainer using standard video alone; they still rely on wearable sensors.

Project Objective

- Develop a user-friendly web application for user management, workout selection, and live training sessions.
- Create a computer vision engine to instantly compare the user's live posture against a trainer's video.
- Deliver immediate visual feedback during workouts to correct the user's form as mistakes happen.
- Generate a detailed performance summary after each session, highlighting achievements and areas for improvement.
- Implement personalized dashboards for different users to track their long-term progress and historical data.

Purpose and Need

- Home based workouts demand intelligent monitoring solutions.
- Access to professional trainers is not always feasible for every individual.
- Many fitness applications operate only on predefined workouts and do not adapt to varied exercise types.
- The platform provides meaningful feedback for any repetition based exercise.
- Exercise performance varies in speed and tempo, requiring robust analytical systems.
- The system accurately counts repetitions across various types of workouts.
- The platform stores historical workout data for performance analysis and improvement tracking.

Literature Survey

Viewpoint-Invariant Exercise Repetition Counting Using Skeleton Data
(Y. C. Hsu, C. Y. Chen, and Y. L. Lin, 2023)

Overview
- Introduces a skeleton-based repetition counting method robust to camera viewpoint variations.
- Uses body joint coordinates instead of raw video frames.
- Designed for flexible exercise monitoring in different environments.

Advantages
- Maintains accuracy across different camera angles.
- Skeleton representation reduces background noise.
- Improves consistency across recording setups.
- Lower computational complexity compared to image-based methods.

Disadvantages
- Highly dependent on accurate pose estimation.
- Skeleton tracking errors affect counting performance.
- Limited validation on highly dynamic exercises.

Pūioio: On-Device Real-Time Smartphone-Based Automated Exercise Repetition Counting System
(A. Sinclair, K. Kautai, and S. R. Shahamiri, 2023)

Overview
- Presents a smartphone-based system for automatic exercise repetition counting.
- Combines pose estimation with motion-state detection.
- Designed for real-time mobile fitness monitoring applications.

Advantages
- Real-time performance on mobile devices.
- No need for specialized hardware or sensors.
- Lightweight and suitable for fitness applications.
- Easily deployable for home workout monitoring.

Disadvantages
- Accuracy decreases during rapid movements.
- Sensitive to lighting and background conditions.
- Limited assessment of exercise quality or correctness.

Recognizing Exercises and Counting Repetitions in Real Time
(T. Alatiah and J. Choi, 2020)

Overview
- Proposes a real-time system capable of recognizing exercises while counting repetitions simultaneously.
- Integrates pose estimation with machine learning classification.
- Intended for intelligent fitness coaching systems.

Advantages
- Simultaneous exercise recognition and rep counting.
- Provides real-time workout monitoring.
- Suitable for virtual coaching and rehabilitation systems.
- Reduces manual tracking effort.

Disadvantages
- Difficulty handling irregular repetition speeds.
- Requires controlled recording conditions.
- Limited robustness to motion variation among users.

Recognition and Repetition Counting for Complex Physical Exercises Using Deep Learning
(A. Soro, S. Brunner, and R. Wattenhofer, 2019)

Overview
- Proposes a deep learning–based framework for automatic recognition and repetition counting of physical exercises.
- The system analyzes temporal body motion patterns extracted from exercise sequences.
- Designed to monitor complex gym exercises without manual supervision.
- Evaluated on multiple cyclic workouts such as squats, push-ups, and sit-ups.

Advantages
- Achieves high repetition counting accuracy across multiple exercise categories.
- Deep learning captures temporal motion dynamics effectively.
- Performs reliably for repetitive and structured movements.
- Demonstrates good generalization across different users and execution styles.
- Enables automated fitness monitoring applications.

Disadvantages
- Requires large annotated datasets for effective model training.
- Performance degrades when body joints are partially occluded.
- Highly dependent on pose estimation accuracy.
- Computational complexity limits deployment on low-power or embedded devices.

Workout Type Recognition and Repetition Counting with Wearable Sensors
(K. Skawinski, P. Biecek, and J. M. Jasiński, 2019)

Overview
- Uses wearable IMU sensors to recognize workout types and count repetitions.
- Motion signals captured directly from body-mounted sensors.
- Applied in sports analytics and rehabilitation monitoring.

Advantages
- High accuracy due to direct motion sensing.
- Independent of lighting and camera positioning.
- Works reliably even under visual occlusion.
- Effective for fast and dynamic exercises.

Disadvantages
- Requires additional wearable hardware.
- Less convenient than vision-based approaches.
- Sensor placement strongly affects performance.
- Increased setup complexity and cost

Proposed Method

The proposed system is structured as a four-stage motion analysis pipeline, designed to convert raw exercise videos into structured corrective feedback.

Stage 1: Motion Acquisition and Representation

In this stage, both the trainer and user exercise videos are processed.

- Pose estimation extracts skeletal joint coordinates.
- These coordinates are transformed into joint angle representations.
- The output is a set of angle–time signals describing body movement.

This stage converts raw video into structured kinematic data.

Stage 2: Signal Conditioning

The extracted angle signals are refined to ensure stability and consistency.

- Noise reduction improves signal reliability.
- Temporal normalization ensures uniform frame representation.
- Statistically dominant joints are identified using a weighted importance method.

This stage prepares clean and meaningful motion signals for analysis.

Stage 3: Motion Analysis

In this stage, movement patterns are analyzed.

- Repetition boundaries are detected using primary motion-driving joints.
- Each repetition is segmented independently.
- User motion is aligned with trainer motion for deviation analysis.

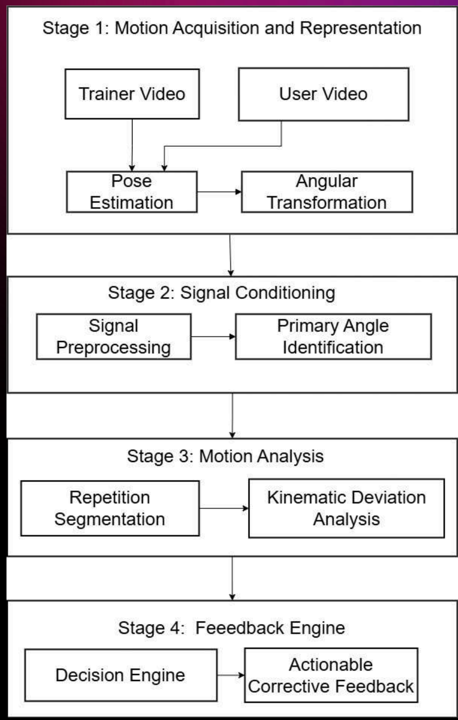
This stage transforms motion signals into structured performance metrics.

Stage 4: Feedback Engine

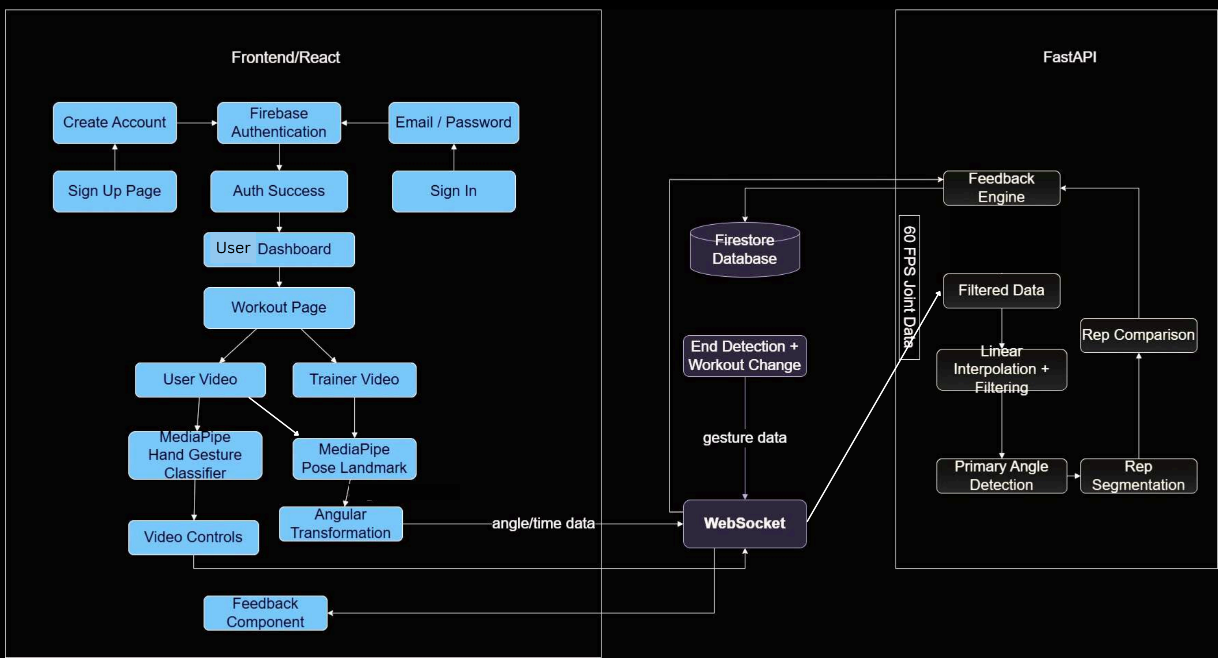
The final stage converts analytical results into actionable insights.

- Deviation metrics are evaluated.
- Joint-specific corrections are generated.
- Repetition count and timestamps are reported.

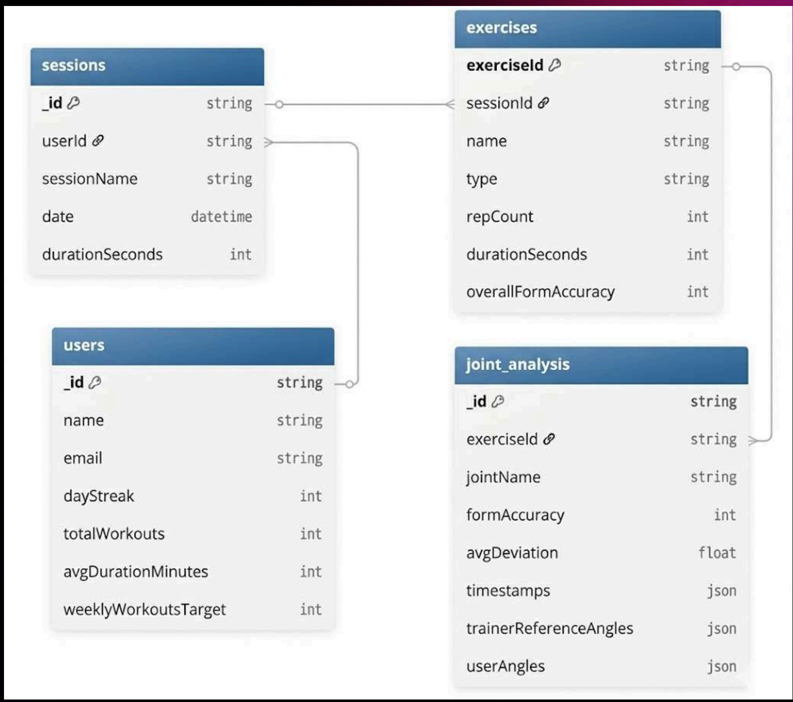
This stage bridges motion analysis with practical performance feedback.



Architecture Diagram



Database Schema



Module 1: User Dashboard and Gestures

Module 1 serves as the primary interaction layer between the user and the posture correction system.

It is responsible for:

- User authentication and session management
- Gesture controlled workout interaction
- Performance analytics visualization
- Workout execution interface
- Detailed exercise-level analysis
- Feedback presentation

React-Based Frontend Architecture :

- The system uses React as a component-based frontend framework to implement modular UI elements. State management enables dynamic rendering of session data and analytics.

Firebase Authentication :

- Firebase Authentication is used for secure user login and registration. It ensures session persistence, user-specific data isolation, and scalable multi-user management.

Firestore Database Integration :

- Cloud Firestore stores structured workout data including session records, repetition counts, timestamps, and feedback results. Data is indexed per authenticated user.

Workout Page Component :

- The Workout Page handles exercise execution, camera interaction, gesture-based video control, and communication with the motion processing backend. Session results are pushed to Firestore after analysis.

User Dashboard Component & Data Synchronization :

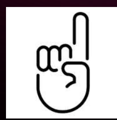
- The Dashboard retrieves workout history and detailed exercise analytics from Firestore. The connection between React components and Firebase services is handled through real-time listeners and asynchronous API calls.

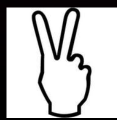
MediaPipe Gesture Detection:
Enable hands-free control of workout sessions using real-time gesture recognition.

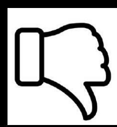
Working :

- Webcam video stream captured
- MediaPipe detects hand landmarks (21 keypoints)
- Landmark coordinates extracted per frame
- Rule-based logic classifies gestures
- Corresponding session control action triggered

Gesture Mapping :

Play : 

Pause : 

Next Exercise:  + 

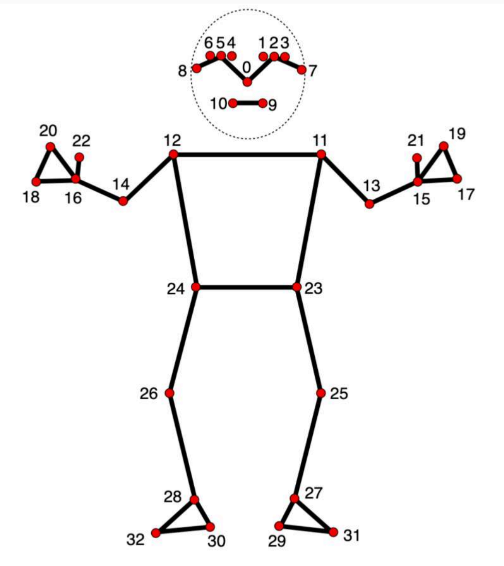
Module 2: Pose Estimation and Angular Transformation

Module 2 performs real-time pose extraction, angular transformation, and signal conditioning to convert raw user and trainer motion data into stable, analysis-ready kinematic signals. This includes:

- 3D landmark extraction using MediaPipe Pose
- Angular transformation (landmarks → joint angles)
- Signal filtering for noise reduction
- Temporal normalization using linear interpolation
- Batch-wise backend transmission

Pose Estimation using MediaPipe Pose:

- MediaPipe Pose model is used for real-time skeletal tracking.
- Extracts 33 3D landmark coordinates (x, y, z) for major body joints.
- Processes both user and trainer video streams independently.
- Frame-wise landmark detection ensures temporal motion continuity.
- Provides structured spatial representation of human posture.



Angular Transformation :

From MediaPipe Pose, each joint i at frame t is represented as a 3D coordinate:

$$Pi(t)=(x_i(t),y_i(t),z_i(t)$$

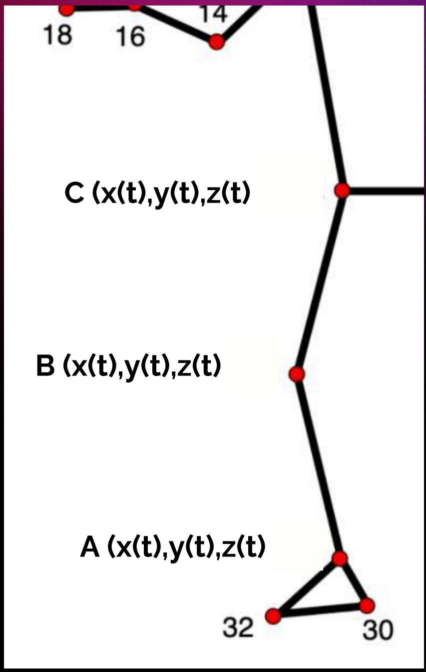
For a joint angle at vertex B, formed by joints A,B,C :

$$BA=(x_A-x_B, y_A-y_B, z_A-z_B)$$

$$BC=(x_C-x_B, y_C-y_B, z_C-z_B)$$

Joint Angle Computation :

$$\theta(t) = \cos^{-1}\left(\frac{\vec{BA}\cdot\vec{BC}}{|\vec{BA}|\,|\vec{BC}|}\right)$$



Backend Transmission :

- For each frame, all computed joint angles of both user and trainer are arranged according to timestamp.
- Each frame contains synchronized user angles, trainer angles, and gesture data.
- Frame-level data is sequentially structured before transmission.
- One batch corresponds to 1 second of motion data (all frames within one second based on FPS).
- Data is transmitted to the backend via WebSockets for continuous, real-time communication.
- Each batch contains complete joint angle and gesture information for uninterrupted processing.
- Batch-wise WebSocket transmission ensures low latency, controlled memory usage, and real-time responsiveness.

Hybrid Incremental Filtering :

- The system applies a Hybrid Incremental Filter for noise reduction.
- The hybrid filter consists of two stages:
  - Kalman Filter – Removes high-frequency sensor noise and landmark jitter.
  - Gaussian Rolling Window Filter – Smooths motion trajectories while preserving movement trends.
- Filtering is performed incrementally without reprocessing historical data.
- Separate filter instances are maintained for:
  - Raw angle data
  - 60 FPS interpolated angle data
- This ensures real-time performance with minimal memory overhead.

Linear Interpolation :

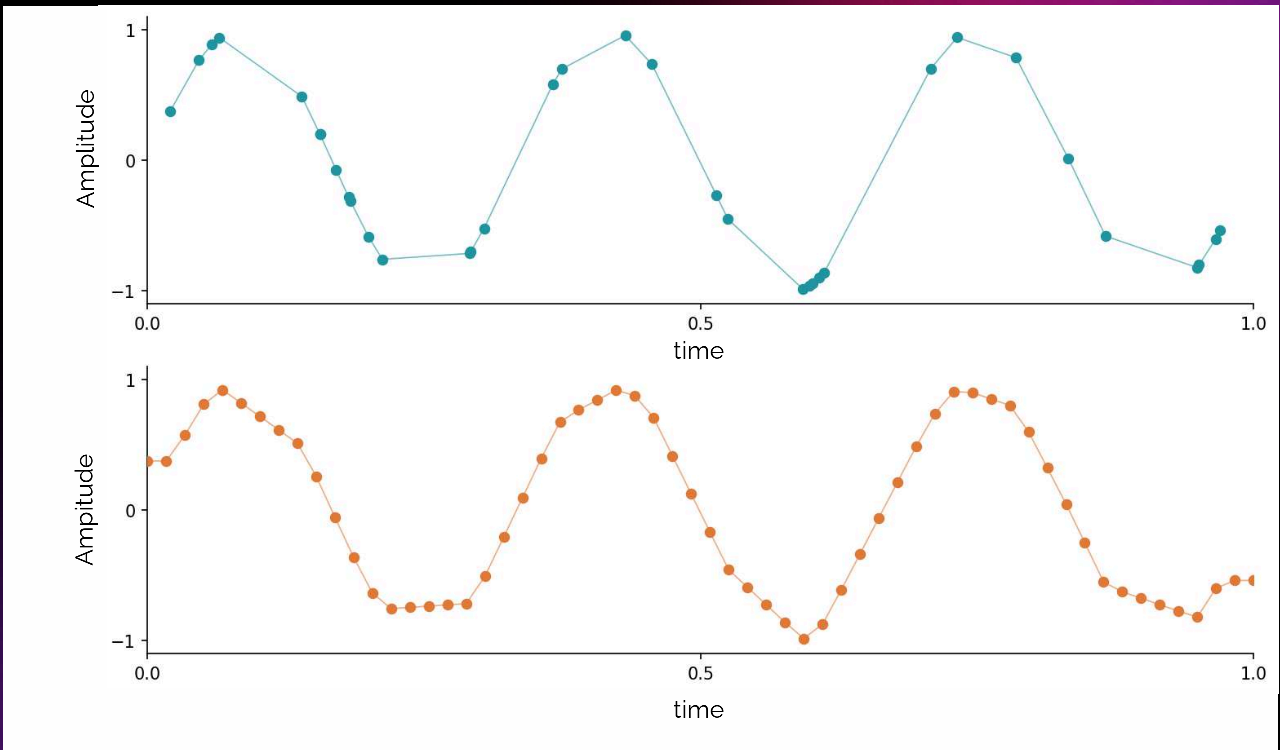
- After filtering, angle signals may have irregular timestamps due to variable frame rates.
- Linear interpolation is applied to resample the angle data to a fixed 60 FPS.

Given two known points: $(t_1, \theta_1), (t_2, \theta_2)$

For any intermediate timestamp t, where, $t_1 \leq t \leq t_2$: $\theta(t) = \theta_1 + \frac{t - t_1}{t_2 - t_1}(\theta_2 - \theta_1)$

Advantages

- Ensures constant temporal resolution (exactly 60 frames per second).
- Eliminates inconsistencies caused by variable camera FPS.
- Enables fair and synchronized comparison between user and trainer signals.
- Produces stable, evenly spaced time-series data for statistical analysis.



Module 3 : Primary Angle Detection and Rep Segmentation

Module 3 converts filtered joint angle signals into structured repetition events using statistical selection and temporally-aware deep learning.

- Identifies dominant motion-driving joints using statistical variance analysis.
- Selects primary angles to focus on biomechanically significant movement.
- Applies a U-Net BiLSTM model for frame-wise phase classification (Rest / Phase 1 / Phase 2).
- Segments motion into structured repetitions based on phase transitions.
- Outputs repetition count and precise start–end timestamps for further posture analysis.

Primary Angle Selection :
Step 1: Motion Variability Measurement
For each joint angle $\theta_i(t)$, compute standard deviation:

$$\sigma_i = \sqrt{\frac{1}{N}\sum_{t=1}^{N}(\theta_i(t) - \mu_i)^2}$$

where σ_i represents movement intensity

- Higher σ_i ⇒ greater motion contribution

Step 2: Descending Sorting
All joints are sorted in descending order:

$$\sigma_{(1)} \geq \sigma_{(2)} \geq \cdots \geq \sigma_{(j)}$$

This ranks joints from most active to least active.

Step 3: Maximum Difference Detection :
Compute consecutive differences:

$$\Delta_k = \sigma_{(k)} - \sigma_{(k+1)}$$

Find the largest drop:

$$k^* = \arg\max_k(\Delta_k)$$

All joints above this index are selected as Primary Angles.

- This identifies the natural separation between dominant and non-dominant joints.
- If the maximum standard deviation among all joints is less than 5 degrees, no primary angles are selected.
- This indicates that the movement amplitude is too small to represent a valid exercise repetition and likely corresponds to a rest state or negligible motion.

Primary Joints: 2 / 13

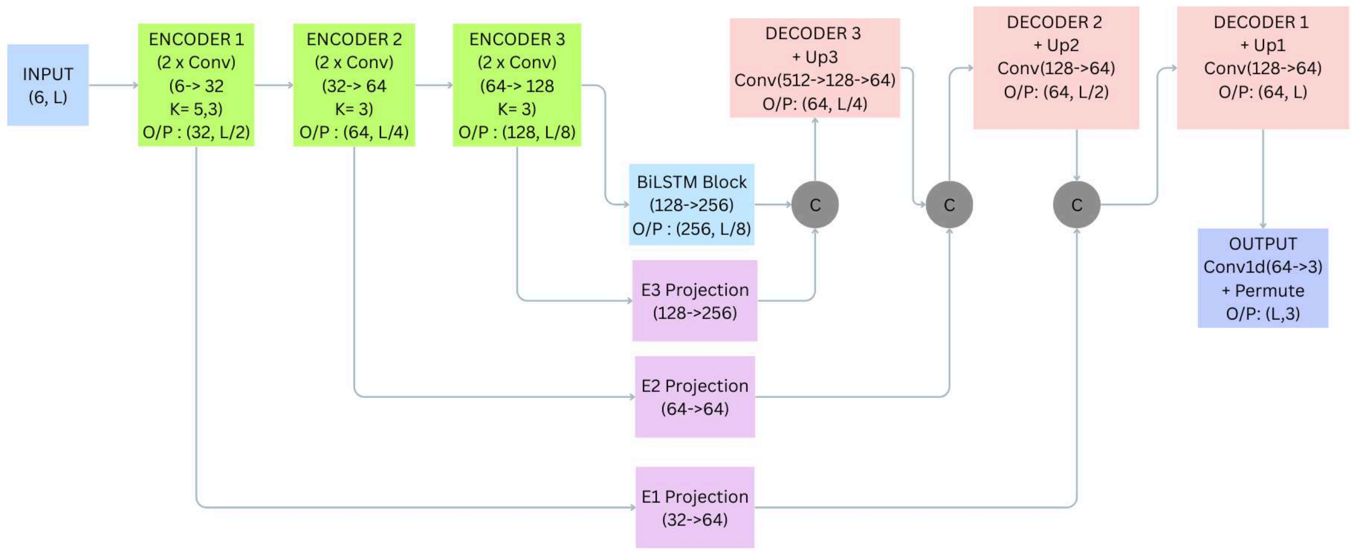
Rank	Joint Name	Std Dev	Cumulative Avg	Data Points	Is Primary
1	Right Elbow	75.08	106.02	258	✓ PRIMARY
2	Left Elbow	70.55	109.53	258	✓ PRIMARY
3	Right Knee	19.91	166.62	258	
4	Neck	19.51	74.45	258	
5	Right Hip	18.88	155.44	258	
6	Right Spine	18.88	155.44	258	
7	Left Knee	18.78	167.35	258	
8	Left Hip	17.42	157.74	258	
9	Left Spine	17.42	157.74	258	
10	Right Ankle	7.12	167.59	258	
11	Left Shoulder	5.89	17.41	258	
12	Right Shoulder	5.59	18.68	258	
13	Left Ankle	4.72	170.88	258	

U-Net + BiLSTM

- Input: 6 joint signals over time
- Task: Classify each frame into:
  - Rest
  - Phase 1 (First half of rep)
  - Phase 2 (Second half of rep)
- Architecture Type:
  - 1D U-Net (spatial feature extraction)
  - BiLSTM (temporal modeling)
  - Skip connections (feature preservation)

Disadvantages

- Continuous biomechanical motion was forced into discrete phase labels (0,1,2).
- Transition from Phase 1 → Phase 2 was gradual in reality but abrupt in labeling, creating ambiguity at boundaries.
- Boundary frames were difficult to classify consistently.
- Small dataset limited the model's ability to learn subtle phase differences.
- Class imbalance between rest and active phases affected training stability.
- The U-Net BiLSTM architecture had a large number of trainable parameters.
- High model complexity was not suitable for the limited dataset size.



Module 4 :Rep Comparison & Feedback System

- This module checks how well a user performs each exercise repetition.
- It compares the user's primary-joint angle sequence with an averaged trainer reference repetition.
- The user rep and the averaged trainer reference rep are time-aligned so they can be compared frame by frame.
- The system measures:
  - posture accuracy
  - movement completion (range of motion)
  - execution speed
- These measurements are converted into normalized scores and categories.
- The weakest performing metrics are identified, and based on them, the system generates rep-level feedback for the user, classified as:
  - Excellent
  - Fair
  - Critical

Reference Rep Construction

Purpose: Build a stable trainer reference repetition and align each user repetition to it before comparison.

Step 1 – Detected reps and primary-joint angle array

- From pose estimation, extract the primary-joint angle per frame.
- Each repetition is a 1-D sequence: $A = [a_1, a_2, \ldots, a_T]$

Step 2 – Averaged trainer reference rep (trainer)

- All trainer repetitions are first resampled to the same length and then averaged:

$$R_{\text{ref}}(t) = \frac{1}{N}\sum_{i=1}^{N} R_i(t)$$

Where

- Ri(t) → i-th trainer rep
- N → number of trainer reps
- This avoids noise and timing variation from a single rep.

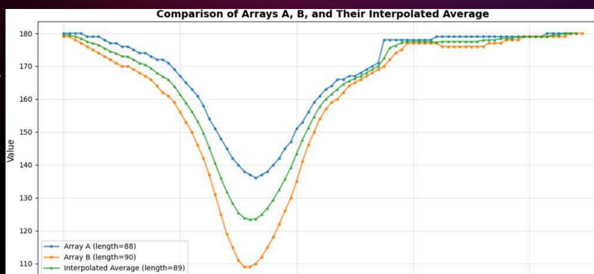
Rep Alignment

Step 3 – Rep length normalization (time alignment)

- Both reference and user reps are resampled to a common length L: $R_{\text{ref}} \to L, \qquad U_{\text{rep}} \to L$

Step 4 – Aligned signals for comparison:

- Reference reps: $R = [r_1, \ldots, r_L]$
- User reps: $U = [u_1, \ldots, u_L]$
- These aligned angle sequences are used for all metric computations.



Error Computation and Core Metric Extraction

- After temporal alignment, the system computes the frame-wise angle error between the user rep and the trainer reference rep.
- From this error signal, the following core posture metrics are extracted:
  - Mean Absolute Error (MAE)
  - Root Mean Square Error (RMSE)
- Maximum deviation
- In parallel, movement completion is evaluated using:
  - user and trainer Range of Motion (ROM)
- Execution tempo similarity is evaluated using:
  - rep length ratio (speed metric)

Posture Accuracy Metrics (Angle-based Similarity)

1. Mean Absolute Error (MAE)
   - Measures the average angular difference between user and trainer across all frames of a rep.
   - It shows the overall posture accuracy.

Formula

$$MAE = \frac{1}{T}\sum_{t=1}^{T}|\theta_u(t) - \theta_r(t)|$$

- T→ number of frames
- $\theta u(t)$ → user angle
- $\theta r(t)$ → reference (trainer average) angle

Interpretation

- Smaller MAE → posture is closer to trainer.

Thresholds (example)

- MAE ≤ 5° → Excellent
- 5° < MAE ≤ 10° → Fair
- MAE > 10° → Critical

Posture Accuracy Metrics (Angle-based Similarity)

2. Root Mean Square Error (RMSE)

- Measures how strongly large mistakes are penalized.
- It is more sensitive to big errors than MAE.

Formula:

$$RMSE = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(\theta_u(t) - \theta_r(t))^2}$$

Interpretation

- High RMSE means the user had large posture deviations at some moments.

Thresholds

- RMSE ≤ 7° → Excellent
- 7° < RMSE ≤ 12° → Fair
- RMSE > 12° → Critical

Posture Accuracy Metrics (Angle-based Similarity)

3. Maximum Deviation

- Measures the worst single-frame error in the repetition.

Formula

$$MaxError = \max_t |\theta_u(t) - \theta_r(t)|$$

Interpretation

- Shows if there was a dangerous or highly incorrect posture at any instant.

Thresholds

- MaxError ≤ 12° → Excellent
- 12° < MaxError ≤ 20° → Fair
- MaxError > 20° → Critical

Posture Accuracy Metrics (Angle-based Similarity)

4. Range of Motion (ROM)

What it measures

- Whether the user completes the full joint movement range compared to the trainer.

Computation

- User ROM = max(θu) − min(θu)
- Trainer ROM = max(θr) − min(θr)
- ROM difference = User ROM − Trainer ROM (evaluated using absolute value

Interpretation

- If ROM is smaller than the trainer, the user is not completing the exercise fully.

Thresholds used for scoring

- |ROM difference| ≤ 10° → Excellent
- 10° < |ROM difference| ≤ 25° → Fair
- |ROM difference| > 25° → Critical

Movement Timing Metric (Speed)

5. Speed (Rep Duration Similarity)

What it measures

- How fast the user performs a repetition compared to the trainer.

Computation

- Speed ratio:

$$r = \frac{\text{user rep length}}{\text{trainer rep length}}$$

- Deviation = |speed ratio − 1|

Interpretation

- Ratio close to 1 means similar tempo.

Thresholds (from your scoring logic)

- Deviation ≤ 0.2 → Excellent
- 0.2 < deviation ≤ 0.5 → Fair
- Deviation > 0.5 → Critical

Overall Score Aggregation & Weak-Area Detection

What happens

- All metric scores are combined using weighted averaging.

Weights used:

$$S_{overall} = 0.20S_{speed} + 0.30S_{mae} + 0.25S_{rmse} + 0.15S_{rom} + 0.10S_{max}$$

Overall category:

- ≥ 80 → Excellent
- 50 – 79 → Fair
- < 50 → Critical

Weak-area identification:

- Any metric with score < 60 is marked as a weak area
- A score below 60 is considered a weak area because it represents a noticeable deviation and is close to the Critical performance region, so it is used as an early-warning threshold for feedback generation.
- The lowest three scores are selected.

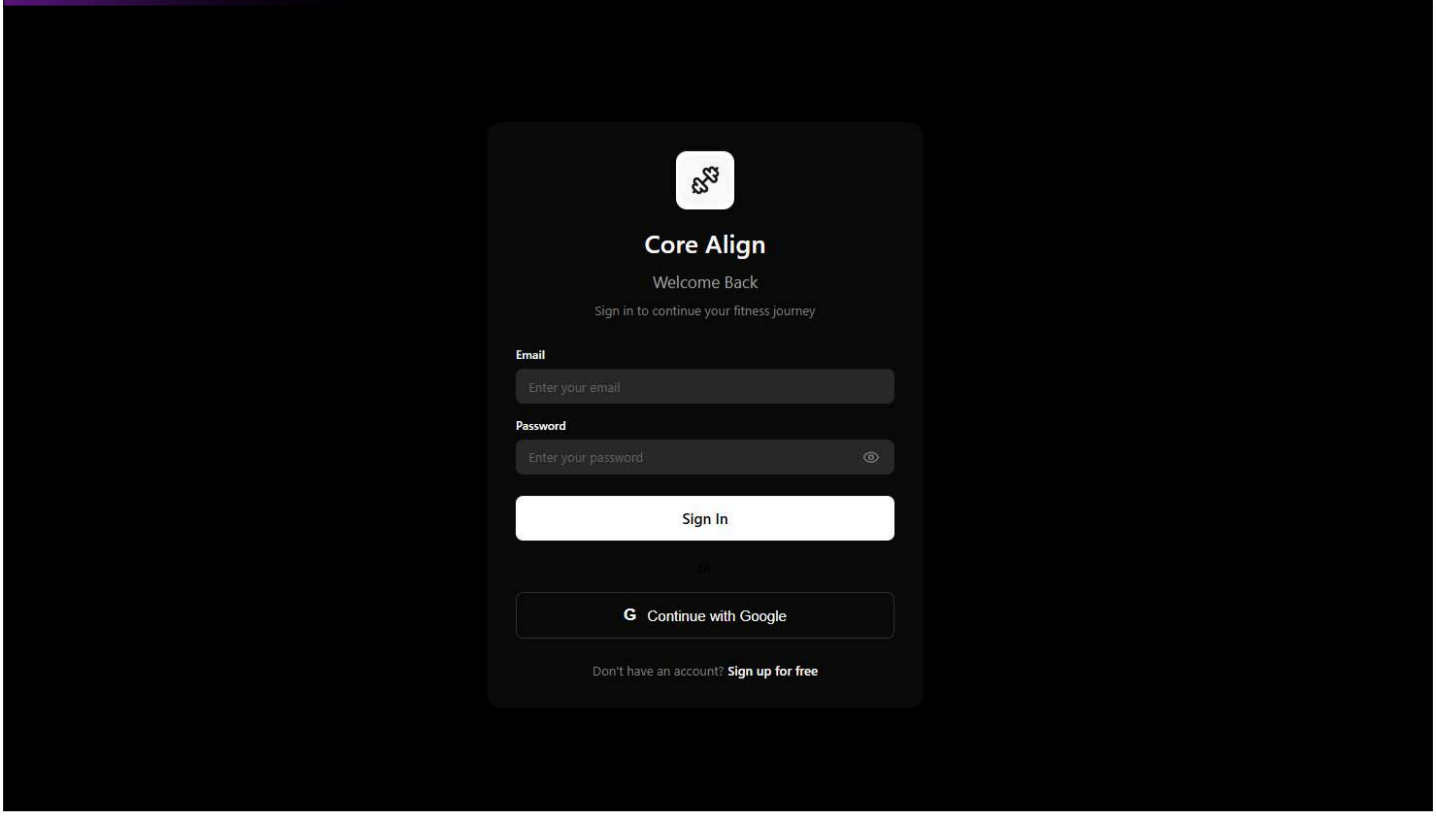
Feedback Generation Logic

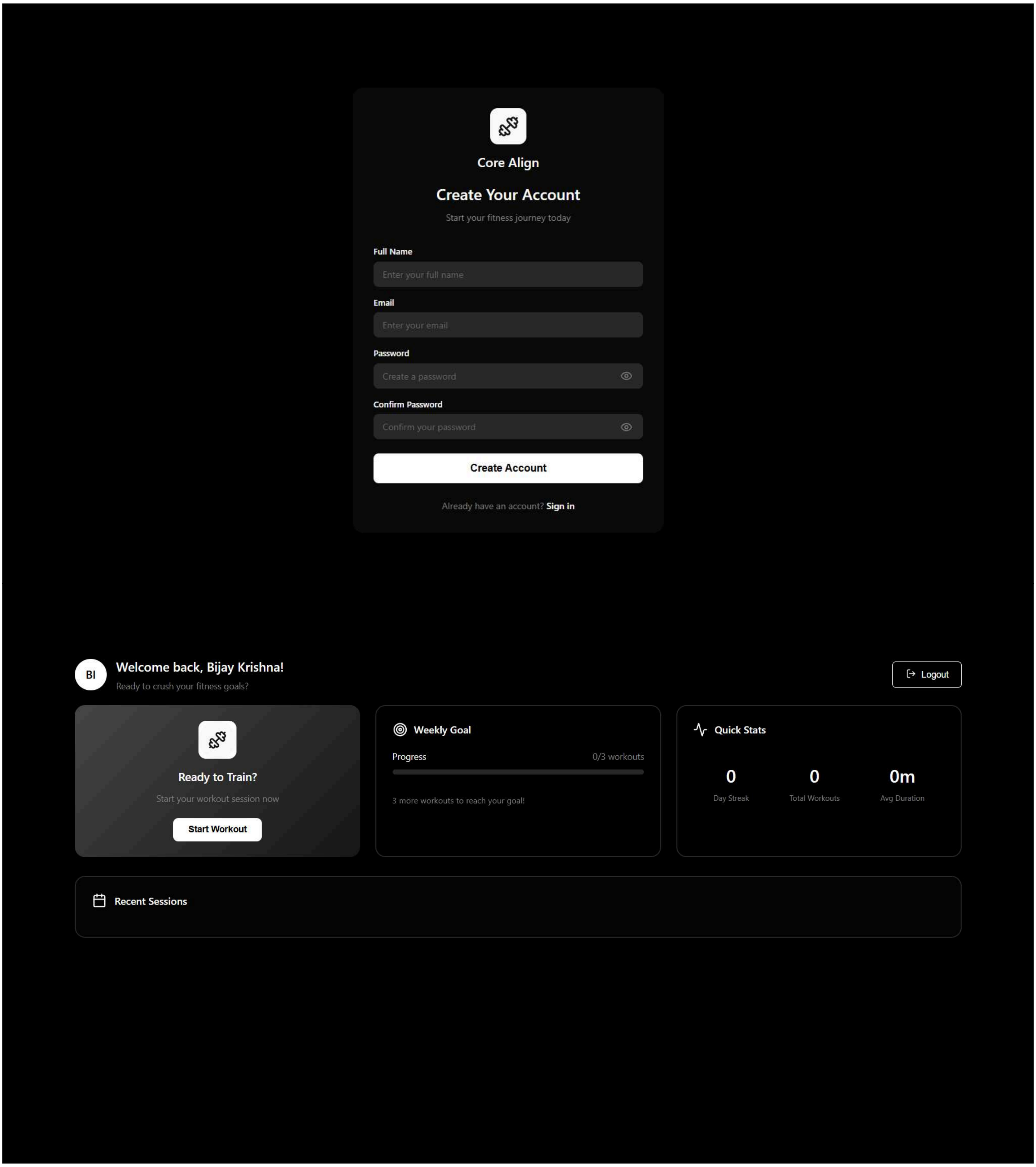
- Each repetition is evaluated independently using the rep-comparison module.
- After every ~5 seconds, the latest repetition (or sufficient motion segment) is analysed.
- Weak metrics (low-scoring metrics) are identified and only the most important issues are selected.

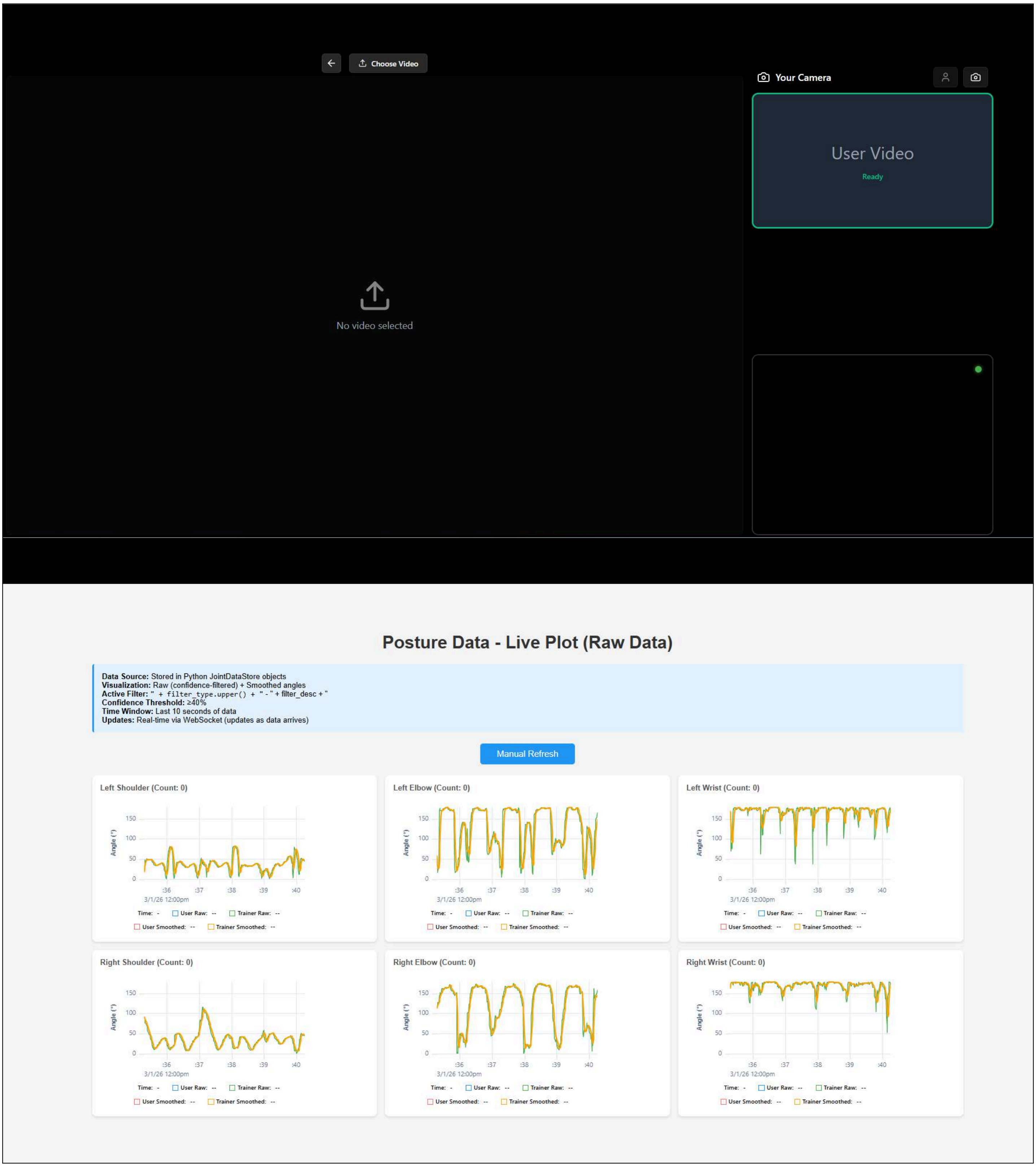
Targeted feedback mapping

- ROM → Critical
- → Increase movement depth and complete the full range of motion.
- RMSE or Maximum deviation → Critical
- → Correct joint alignment during peak movement.
- Speed → Critical
- → Maintain a tempo closer to the trainer.

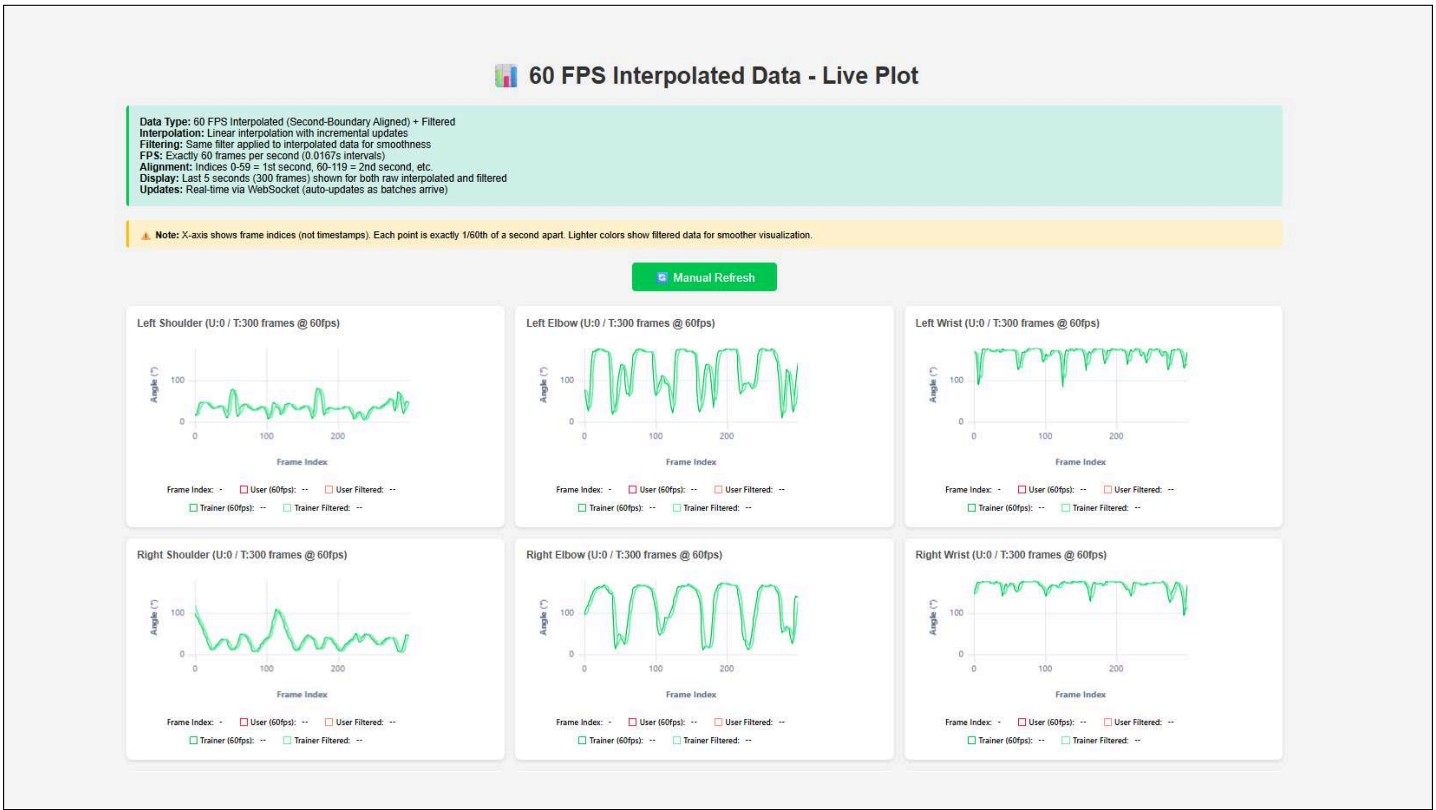
RESULTS AND OUTPUT

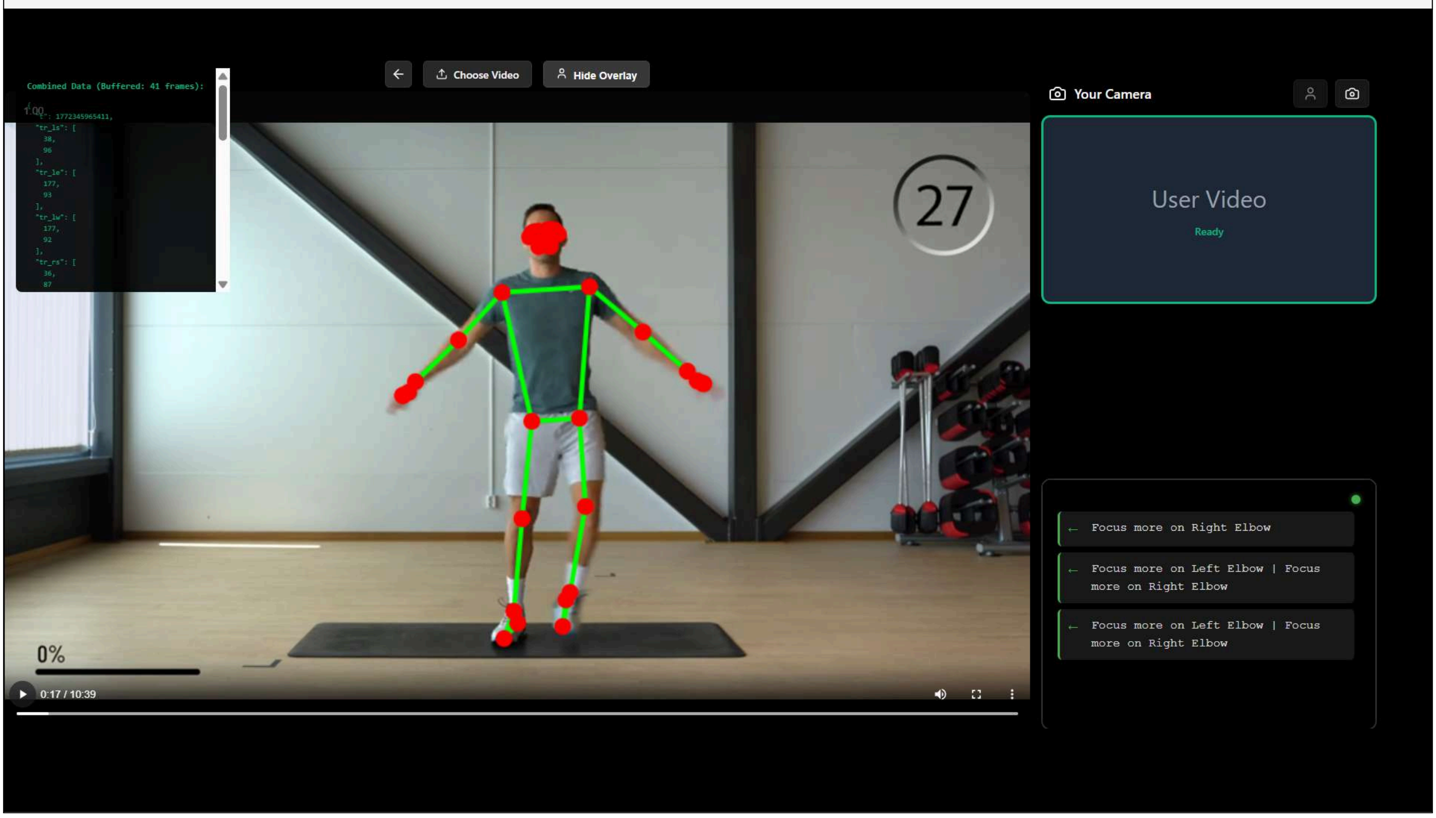


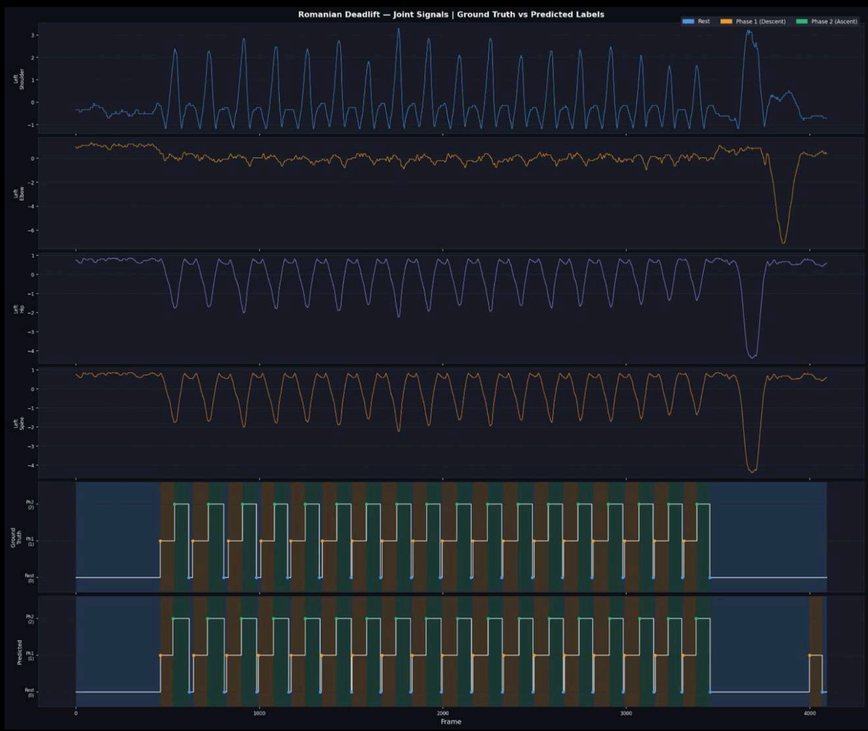

Core Align
Create Your Account
Start your fitness journey today
Full Name
Enter your full name
Email
Enter your email
Password
Create a password
Confirm Password
Confirm your password
Create Account
Already have an account? Sign in
BI
Welcome back, Bijay Krishna!
Ready to crush your fitness goals?
Logout
Ready to Train?
Start your workout session now
Start Workout
Weekly Goal
Progress
0/3 workouts
3 more workouts to reach your goal!
Quick Stats
0
Day Streak
0
Total Workouts
0m
Avg Duration
Recent Sessions

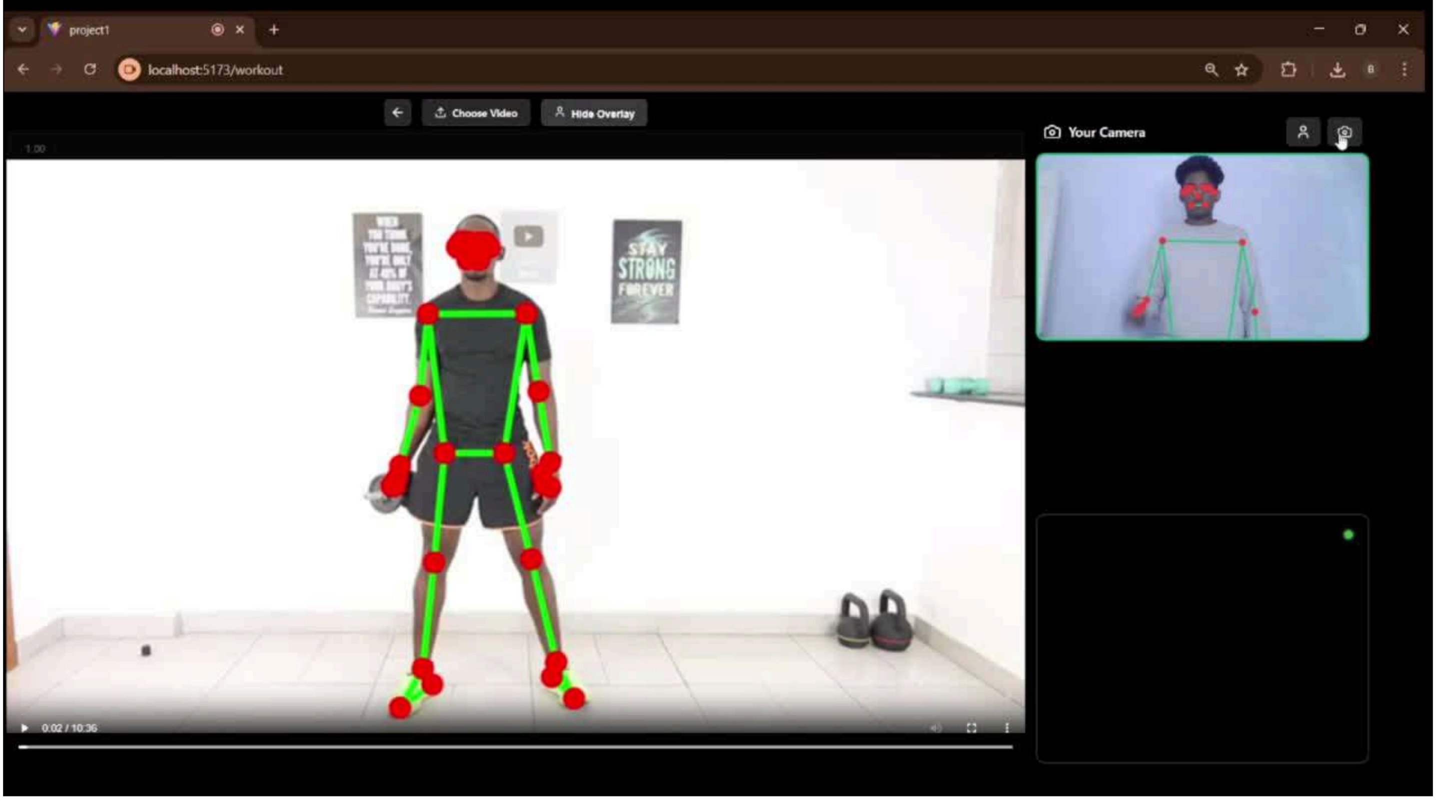


Choose Video
Your Camera
User Video
Ready
No video selected
Posture Data - Live Plot (Raw Data)
Data Source: Stored in Python JointDataStore objects
Visualization: Raw (confidence-filtered) + Smoothed angles
Active Filter: " + filter_type.upper() + " - " + filter_desc + "
Confidence Threshold: ≥40%
Time Window: Last 10 seconds of data
Updates: Real-time via WebSocket (updates as data arrives)
Manual Refresh
Left Shoulder (Count: 0)
Left Elbow (Count: 0)
Left Wrist (Count: 0)
Right Shoulder (Count: 0)
Right Elbow (Count: 0)
Right Wrist (Count: 0)
Angle (°)
3/1/26 12:00pm
Time: -
User Raw: --
Trainer Raw: --
User Smoothed: --
Trainer Smoothed: --


60 FPS Interpolated Data - Live Plot
Data Type: 60 FPS Interpolated (Second-Boundary Aligned) + Filtered
Interpolation: Linear interpolation with incremental updates
Filtering: Same filter applied to interpolated data for smoothness
FPS: Exactly 60 frames per second (0.0167s intervals)
Alignment: Indices 0-59 = 1st second, 60-119 = 2nd second, etc.
Display: Last 5 seconds (300 frames) shown for both raw interpolated and filtered
Updates: Real-time via WebSocket (auto-updates as batches arrive)
Note: X-axis shows frame indices (not timestamps). Each point is exactly 1/60th of a second apart. Lighter colors show filtered data for smoother visualization.
Manual Refresh
Left Shoulder (U:0 / T:300 frames @ 60fps)
Left Elbow (U:0 / T:300 frames @ 60fps)
Left Wrist (U:0 / T:300 frames @ 60fps)
Right Shoulder (U:0 / T:300 frames @ 60fps)
Right Elbow (U:0 / T:300 frames @ 60fps)
Right Wrist (U:0 / T:300 frames @ 60fps)
Angle (°)
Frame Index
Frame Index: - User (60fps): -- User Filtered: --
Trainer (60fps): -- Trainer Filtered: --


Combined Data (Buffered: 41 frames):
Choose Video
Hide Overlay
Your Camera
User Video
Ready
27
0%
0:17 / 10:39
Focus more on Right Elbow
Focus more on Left Elbow | Focus more on Right Elbow
Focus more on Left Elbow | Focus more on Right Elbow


Romanian Deadlift — Joint Signals | Ground Truth vs Predicted Labels
Rest
Phase 1 (Descent)
Phase 2 (Ascent)
Frame


project1
localhost:5173/workout
Choose Video
Hide Overlay
Your Camera
STAY STRONG FOREVER
0:02 / 10:36

Assumptions

- MediaPipe provides sufficiently accurate and temporally stable 3D body keypoints for joint-angle computation.
- The trainer video represents the reference (ideal) execution for each exercise.
- The user performs the exercise under reasonably controlled and acceptable conditions, including adequate lighting, appropriate camera placement, and a stable posture setup.
- Full body of both trainer and user is visible with minimal self-occlusion and no major external occlusions.

Role Breakdown and Contributions

Aaron Joseph : User Dashboard & Gesture Interaction

Responsibilities

- Design and develop the React-based user interface
- Implement user authentication and session management using Firebase
- Build the Workout Dashboard and analytics visualization
- Enable gesture-controlled workout navigation
    - Play / Pause / Next Exercise
- Integrate webcam interaction with workout execution
- Display real-time feedback received from backend modules

Key Contributions

- User login & session handling
- Dashboard data synchronization with Firestore
- Gesture-based hands-free workout control
- Feedback visualization for users

Role Breakdown and Contributions

Bijay Krishna D : Pose Estimation & Motion Processing

Responsibilities

- Extract 33 body landmarks using MediaPipe Pose
- Convert landmark coordinates into joint angles
- Perform signal preprocessing
  - Noise filtering
  - Interpolation
  - Temporal normalization
- Handle real-time video processing
- Structure motion data and send batches via WebSockets

Key Contributions

- Stable kinematic signal generation
- Angular transformation algorithms
- Motion data preparation for analysis
- Real-time pose tracking pipeline

Role Breakdown and Contributions

Abin J Arackal : Repetition Counting & Movement Analysis

Responsibilities

- Identify primary moving joints dynamically
- Normalize exercise cycles
- Prepared and train the dataset
- Implement intelligent rep segmentation model
- Segment each repetition for further comparison

Key Contributions

- Noise-resistant rep detection
- Dynamic calibration based on user movement
- Accurate real-time repetition counting
- Rep segmentation for evaluation

Role Breakdown and Contributions

Angelo Varghese: Comparison & Feedback System

Responsibilities

- Generate ideal reference movement from trainer data
- Compare user reps using
- Measure deviation in reps
- Detect form mistakes automatically
- Generate real-time corrective feedback
- Send feedback to dashboard for display

Key Contributions

- Intelligent posture comparison
- Exercise-agnostic evaluation system
- Personalized correction feedback
- Final decision-making module of the system

Hardware and Software Requirements

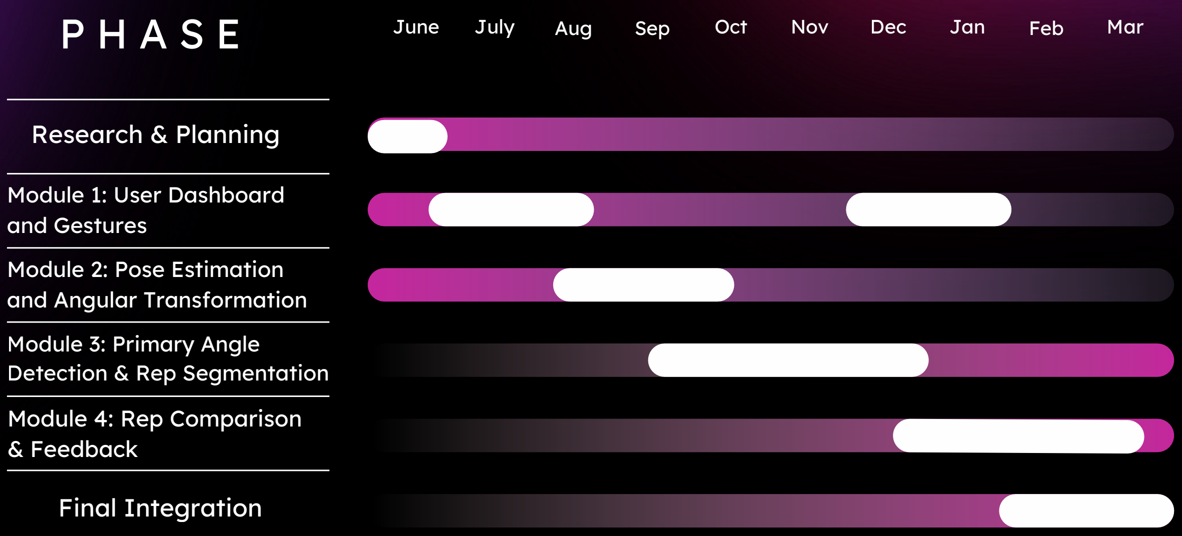
Software Requirements

- Frontend Framework: React.js
- Backend Framework: FastAPI (Python)
- Real-Time Communication: WebSocket
- Video Processing: MediaPipe Pose, MediaPipe
- Database: Firebase

Hardware Requirements

- Processor: Intel i5 (10th Gen or above) / AMD Ryzen 5 or better
- RAM: Minimum 8 GB (Recommended: 16 GB for smooth rendering)
- GPU : 4GB VRAM or above
- Webcam: Required (for user pose capture)
- OS: Windows 10/11, macOS, or modern Linux
- Browser: Latest version of Chrome, Edge, or Firefox (for TF.js compatibility and screen capture)

Gantt Chart



Risk and Challenges

Challenges :

- Continuous and back-to-back repetitions make accurate rep boundary detection and cycle segmentation difficult.
- Validating and tuning the real-time feedback logic while the user is actively exercising is challenging due to latency, motion noise, and user variability.
- Absence of a standardized, labelled trainer–user paired video dataset makes quantitative evaluation and benchmarking of posture comparison accuracy difficult

Risks :

- Real-time processing might lag on low-end mobile devices.
- Accuracy of pose comparison can vary with lighting or occlusion.
- Repition detected by the model need not be always accurate

Conclusion

- This presentation has outlined our comprehensive design for an AI-powered posture analysis system, establishing a clear architectural blueprint.
- The system's key innovation lies in its proposed exercise-agnostic approach, which will allow it to dynamically analyze any repetitive motion and offer a uniquely flexible and intelligent feedback.
- The proposed modular architecture establishes a robust and scalable foundation, ensuring that the final product can be easily expanded with more advanced analytics and features in the future.

References

- [1] A. Soro, S. Brunner, and R. Wattenhofer, "Recognition and repetition counting for complex physical exercises using deep learning," Sensors, vol. 19, no. 3, p. 714, 2019.
-
- [2] Y. C. Hsu, C. Y. Chen, and Y. L. Lin, "Viewpoint-invariant exercise repetition counting using skeleton data," IEEE Access, vol. 11, pp. 123456–123468, 2023.
-
- [3] A. Sinclair, K. Kautai, and S. R. Shahamiri, "Pūioio: On-device real-time smartphone-based automated exercise repetition counting system," arXiv preprint arXiv:2308.02420, 2023.
-
- [4] T. Alatiah and J. Choi, "Recognizing exercises and counting repetitions in real time," arXiv preprint arXiv:2005.03194, 2020.
-
- [5] K. Skawinski, P. Biecek, and J. M. Jasiński, "Workout type recognition and repetition counting with wearable sensors," in Proc. Ambient Intelligence Workshop, 2019

References

- [6] B. Ferreira, J. Carvalho, and A. Rocha, “Deep learning approaches for workout repetition counting and validation,” Pattern Recognition Letters, vol. 145, pp. 1–8, 2021.
- [7] F. Japhne, M. Rahman, and S. Lee, “FitCam: Detecting and counting repetitive exercises using deep learning,” Journal of Big Data, vol. 11, no. 1, 2024.
- [8] Y. Tang, H. Liu, and J. Wang, “MultiCounter: Multiple action-agnostic repetition counting in untrimmed videos,” arXiv preprint arXiv:2409.04035, 2024.
- [9] Y. Lim and S. Lee, “Intelligent repetition counting for unseen exercises using few-shot learning,” arXiv preprint arXiv:2410.00407, 2024.
- [10] J. Liu, X. Zhang, and L. Chen, “YOLO-Fit IoT: Real-time exercise pose analysis and personalized training system,” Alexandria Engineering Journal, vol. 86, pp. 245–257, 2025.

Thank You!

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Programme Specific Outcomes (PSO)
A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills
The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills
The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills
The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes (CO)

Course Outcome 1: Model and solve real world problems by applying knowledge across domains (Cognitive knowledge level: Apply).

Course Outcome 2: Develop products, processes or technologies for sustainable and socially relevant applications (Cognitive knowledge level: Apply).

Course Outcome 3: Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks (Cognitive knowledge level: Apply).

Course Outcome 4: Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms (Cognitive knowledge level: Apply).

Course Outcome 5: Identify technology/research gaps and propose innovative/creative solutions (Cognitive knowledge level: Analyze).

Course Outcome 6: Organize and communicate technical and scientific findings effectively in written and oral forms (Cognitive knowledge level: Apply).

Appendix C:

CO-PO-PSO Mapping

COURSE OUTCOMES (COs)

The following table describes the course outcomes and their corresponding Bloom's Taxonomy level:

SNO	DESCRIPTION	BLOOM'S TAXONOMY LEVEL
1	Model and solve real world problems by applying knowledge across domains.	Apply
2	Develop products, processes or technologies for sustainable and socially relevant application.	Apply
3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.	Apply
4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.	Apply
5	Identify technology/research gaps and propose innovative/creative solutions.	Analyze
6	Organize and communicate technical and scientific findings effectively in written and oral forms.	Apply

MAPPING COURSE OUTCOMES (COs) PROGRAM OUTCOMES (POs) AND PROGRAM SPECIFIC OUTCOMES (PSOs)

The following table demonstrates the mapping between Course Outcomes (COs), Program Outcomes (POs), and Program Specific Outcomes (PSOs):

CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
CO1	2	2	2	1		2	1	1	1	1	1	2	2	2
CO2	2	2	2		2	3	2	1	1	1	1	2	2	
CO3								3	2	2	1			3
CO4		1	2		3	2	3	2	2	3	1		2	2
CO5	2	3	3	1	2					1	3	3		
CO6			2			2	2	2	3	1	1			3

JUSTIFICATIONS FOR CO-PO AND CO-PSO MAPPING

The following table provides the justification for the mapping between each CO and its mapped POs/PSOs:

Mapping	Level	Justification
101003/ CS822U.CO1-PO1	Medium	Applies engineering knowledge, mathematics, and computing fundamentals to model and solve real-world problems.
101003/ CS822U.CO1-PO2	Medium	Requires identification, formulation and analysis of engineering problems before developing solutions.
101003/ CS822U.CO1-PO3	Medium	Involves designing systems or solutions to address identified engineering needs.
101003/ CS822U.CO1-PO4	Low	Limited investigation or experimentation may be required to validate the developed models or solutions.
101003/ CS822U.CO1-PO6	Medium	Solutions developed may influence societal, environmental, or sustainability aspects of engineering applications.
101003/ CS822U.CO1-PO7	Medium	Encourages consideration of ethical and professional responsibilities while developing solutions.
101003/ CS822U.CO1-PO8	Low	Some level of teamwork may be required while working on modeling or project tasks.
101003/ CS822U.CO1-PO9	Low	Requires communication of results and findings through reports or presentations.
101003/ CS822U.CO1-PO10	Low	Involves basic planning and execution of tasks while implementing solutions.
101003/ CS822U.CO1-PO11	Low	Encourages independent learning and adapting new knowledge to solve real-world problems.
101003/ CS822U.CO1-PSO1	Medium	Uses core computer science principles to analyze problems and design appropriate technical solutions.

Mapping	Level	Justification
101003/ CS822U.CO1-PSO2	Medium	Involves algorithm design and application of software development practices to implement solutions.
101003/ CS822U.CO1-PSO3	Medium	Applies computer science fundamentals in research or innovative problem solving for societal needs.
101003/ CS822U.CO2-PO1	Medium	Utilizes engineering knowledge to design and develop products or systems.
101003/ CS822U.CO2-PO2	Medium	Requires analysis of user needs and technological feasibility before development.
101003/ CS822U.CO2-PO3	Medium	Focuses on design and development of solutions addressing real-world requirements.
101003/ CS822U.CO2-PO5	Medium	Requires use of modern engineering and IT tools for system or product development.
101003/ CS822U.CO2-PO6	High	Emphasizes sustainability and societal impact during solution development.
101003/ CS822U.CO2-PO7	Medium	Ethical and professional responsibilities are considered when developing socially relevant technologies.
101003/ CS822U.CO2-PO8	Low	Development activities may require collaboration among team members.
101003/ CS822U.CO2-PO9	Low	Involves communicating design ideas and development outcomes.
101003/ CS822U.CO2-PO10	Low	Requires basic planning and coordination during development tasks.
101003/ CS822U.CO2-PO11	Low	Encourages continuous learning of emerging technologies while building solutions.
101003/ CS822U.CO2-PSO1	Medium	Applies computer science fundamentals to design technically feasible solutions.
101003/ CS822U.CO2-PSO2	Medium	Uses programming and software development practices to implement solutions.
101003/ CS822U.CO3-PSO3	High	Professional collaboration and teamwork support innovative computer science solutions.

Mapping	Level	Justification
101003/ CS822U.CO3-PO8	High	Strongly relates to functioning effectively as a member or leader in multidisciplinary teams.
101003/ CS822U.CO3-PO9	Medium	Requires effective communication among team members during collaboration.
101003/ CS822U.CO3-PO10	Medium	Team leadership involves project coordination and task management.
101003/ CS822U.CO3-PO11	Low	Team interaction supports learning from peers and adapting new approaches.
101003/ CS822U.CO3-PSO3	High	Professional collaboration and teamwork support innovative computer science solutions.
101003/ CS822U.CO4-PO2	Low	Requires analysis and planning before executing project tasks.
101003/ CS822U.CO4-PO3	Medium	Involves designing and implementing project solutions based on planned activities.
101003/ CS822U.CO4-PO5	High	Requires effective use of modern tools and resources during project execution.
101003/ CS822U.CO4-PO6	Medium	Considers societal and environmental impact while implementing solutions.
101003/ CS822U.CO4-PO7	High	Emphasizes adherence to ethical practices and professional responsibilities.
101003/ CS822U.CO4-PO8	Medium	Task execution may involve teamwork and coordination among team members.
101003/ CS822U.CO4-PO9	Medium	Requires communication of progress, outcomes and documentation.
101003/ CS822U.CO4-PO10	High	Focuses on project management including scheduling, resource allocation and task execution.
101003/ CS822U.CO4-PO11	Low	Encourages adaptability and continuous learning during project implementation.
101003/ CS822U.CO4-PSO2	Medium	Uses software development practices to execute and manage project tasks.
101003/ CS822U.CO4-PSO3	Medium	Applies computer science knowledge in practical project development environments.

Mapping	Level	Justification
101003/ CS822U.CO5-PO1	Medium	Uses engineering knowledge to understand technological limitations and opportunities.
101003/ CS822U.CO5-PO2	High	Requires deep problem analysis and identification of research gaps.
101003/ CS822U.CO5-PO3	High	Proposes innovative design solutions to address identified gaps.
101003/ CS822U.CO5-PO4	Low	May involve limited investigation or validation of proposed ideas.
101003/ CS822U.CO5-PO5	Medium	Uses appropriate tools and techniques to analyze and develop innovative solutions.
101003/ CS822U.CO5-PO10	Low	Requires minimal planning while proposing new solution approaches.
101003/ CS822U.CO5-PO11	High	Strongly promotes independent learning and exploration of emerging technologies.
101003/ CS822U.CO5-PSO1	High	Applies computer science fundamentals to identify and solve complex technical problems.
101003/ CS822U.CO6-PO3	Medium	Requires presentation of design concepts and developed solutions.
101003/ CS822U.CO6-PO6	Medium	Communication may include discussion of societal or environmental implications.
101003/ CS822U.CO6-PO7	Medium	Requires responsible and ethical communication of technical results.
101003/ CS822U.CO6-PO8	Medium	Communication supports collaboration within team environments.
101003/ CS822U.CO6-PO9	High	Strongly relates to effective written and oral communication of technical findings.
101003/ CS822U.CO6-PO10	Low	Requires structured reporting and documentation in project management contexts.
101003/ CS822U.CO6-PO11	Low	Encourages continuous improvement in communication and presentation skills.
101003/ CS822U.CO6-PSO3	High	Communication of innovative computer science solutions and research outcomes.